IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

PGARA: Profile-Guided Adaptive Register Allocation via Markov Decision Processes and Game-Theoretic Auctions

SHREYANS CHOWDRY[1], SAMRENDER SINGH SAINI[1], KARTHIKEYA JAMI[1], ABHISHEK KUMAR[1], and NISHANTH KUMAR[1]

[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India (e-mail: shreyans.chowdry2024@vitstudent.ac.in; samrender.singh2024@vitstudent.ac.in; jami.karthikeya2024@vitstudent.ac.in; abhishek.kumar2024b@vitstudent.ac.in); (akula.nishanthkumar2024@vitstudent.ac.in)

Corresponding author: Shreyans Chowdry (e-mail: shreyans.chowdry2024@vitstudent.ac.in).

This work received no external funding.

ABSTRACT The register allocator decides which values a compiler keeps in a processor's few registers, and the right decision depends on how often each path through the program is actually taken. Production allocators rank spill candidates by access counts weighted by loop depth or, with a profile, by block frequency; learned allocators train opaque policies on large corpora. We present PGARA (Profile-Guided Adaptive Register Allocation), which models the residency of every SSA value as a two-state Markov decision process over the control-flow graph with profiled transition probabilities. Policy evaluation yields each value's expected cycles saved under the policy the code generator realises; values on a loop-carried φ-web bid as one coalition; registers are sold in a sealed-bid auction along a maximum-cardinality-search order of the chordal SSA interference graph; and best-response dynamics reach a pure Nash equilibrium. PGARA is embedded in a complete compiler for a C subset, from a DFA lexer and LL(1) parser to SSA optimisation, code generation and a cycle-counting simulator, with a web interface that visualises every phase. On 17 programs, seven of them translated from MiBench and BEEBS, and register budgets from 3 to 32, all 1,428 configurations reproduce the reference output. At four registers PGARA lowers geometric-mean cycles by 21.8% relative to linear scan, 3.7% relative to Chaitin–Briggs colouring with profile-weighted spill costs, 3.3% relative to profile-weighted priority allocation and 2.6% relative to static branch priors, and halves spill traffic relative to linear scan. The benefit is stable across test inputs, confined to budgets of eight registers or fewer, and requires a representative profile.

INDEX TERMS Compiler design, game theory, Markov decision process, profile-guided optimisation, register allocation, static single assignment.

I. INTRODUCTION

COMPILERS are the bridge between the abstractions programmers write and the narrow interface that hardware executes. Their structure has been stable for decades (a front end that recognises and checks the language, a middle end that transforms a machine-independent intermediate representation, and a back end that maps it to instructions and registers), but the techniques inside each phase keep evolving, from reusable multi-level infrastructures such as MLIR [1] to optimisation heuristics learned from data [2]–[4]. Static single assignment (SSA) form [5], [6] is now the common currency of this middle end, and the textbooks used to teach the subject treat it as such [7], [8].

Among back-end problems, register allocation has an outsized effect on performance: a value that is not kept in a register costs a store when it is produced and a load each time it is consumed, and on a loop that runs millions of times these accesses dominate execution time. The problem is hard for two reasons. First, the decision is combinatorial, since values that are live at the same time compete for the same k registers. Second, the payoff of each decision is uncertain at compile time, because it depends on which branches the program actually takes. A value used only on an error path that runs once in a thousand iterations is worth far less than one used on the hot path, yet a static analysis sees the two paths as symmetric. Our motivation is to treat this second difficulty as

a first-class modelling concern rather than a tuning detail.

Existing allocators fall broadly into three families. Graph-colouring allocators, descending from Chaitin [9] and the priority-based variant of Chow and Hennessy [10], build an interference graph and colour it with k colours, choosing spill candidates by a cost that weights each access by an execution-frequency estimate: classically 10^d for loop depth d, and in production compilers such as LLVM's greedy allocator the estimated block frequency, which reflects measured counts when profile-guided optimisation is enabled [11], [12]. Linear-scan allocators [13] trade quality for speed by sweeping live intervals in program order. On SSA form the interference graph is chordal [14], [15], which makes colouring itself polynomial and moves the difficulty entirely into the spilling decision; combinatorial approaches solve allocation and scheduling jointly to proven optimality [16], [17].

A newer family learns allocation and related decisions from data. MLGO [18] trains policies inside LLVM's production allocator, RL4ReAl [19] learns end-to-end register allocation with reinforcement learning over embeddings such as IR2Vec [20], and environments such as CompilerGym [21] and representations such as ProGraML [22] make these experiments reproducible. In parallel, profile-guided post-link optimisers such as BOLT [23] and Propeller [24] have shown that measured execution profiles yield large gains in code layout, and recent work predicts profiles when none are available [25].

These families leave a gap. Spill weights built from static estimates encode a fixed belief about branch behaviour (loops iterate about ten times; both arms of a conditional are equally likely), so they mis-rank values precisely on programs with data-dependent control flow, as early studies of static estimators already observed [26]–[28]. When a profile is available, production compilers already scale spill costs by measured block frequencies [11], [12], and frequency-weighted spill costs go back to hierarchical and progressive allocators [29], [30]. Such profile-weighted costs remove the fixed belief, but they still rank a value by a frequency-weighted count of its accesses: they do not model the value's residency over its live range (where a spill or reload would occur, what calls cost, which loads the peephole pass forwards), they do not make loop-carried φ-webs bid as one, and the heuristic that applies them (simplify, spill, evict) leaves the objective implicit. Whether modelling these effects is worth more than simply weighting accesses by measured frequency is an empirical question that this paper tests directly. Learned policies avoid fixed beliefs, but they require large training corpora and expensive training loops [31], and the decision they make for a particular value cannot be traced back to a quantity a compiler engineer can inspect.

Classical colouring formulations also obscure the objective. A colouring is either feasible or not, and the quality of the result emerges indirectly from the order in which nodes are simplified and spilled. Neither family states, for each value, the expected number of cycles that keeping it in a register will save.

We propose PGARA (Profile-Guided Adaptive Register Allocation), which makes that expected saving the central quantity. It is profile-guided because every probability it uses is measured on a training run, and adaptive because the same program receives a different allocation when its measured behaviour changes, without any hand-tuned weights. Each SSA value's residency is a two-state Markov decision process (MDP) over its live range in the control-flow graph [32], [33]; transition probabilities are measured edge frequencies, and exact policy iteration [34] gives the value's expected gain per invocation. Allocation is then a resource-contention game: values that form a loop-carried φ-web bid together as a coalition, in the spirit of cooperative game theory [35], registers are sold in a sealed-bid auction [36] processed along a maximum-cardinality-search (MCS) order [37] of the chordal interference graph, and best-response dynamics settle the result into a pure Nash equilibrium [38], [39]. The approach is apt because each ingredient matches a structural fact of the problem: SSA makes the contention structure chordal, profiles make the uncertainty measurable, and the gain is an explicit, inspectable number.

The advantages are practical as well as conceptual. Policy iteration on a two-state MDP per value converges in about 1.3 improvement rounds on average, so allocation takes milliseconds; every decision can be explained by a bid and an auction round; and the same engine runs unchanged on static priors, which isolates the value of profiling. To evaluate it honestly we built a complete compiler around it, from a formally specified grammar to a cycle-counting simulator, and a web interface that renders every intermediate artefact.

Contributions. The main contribution is the gain formulation: modelling the register residency of each SSA value as a two-state MDP over its live range, with measured branch probabilities, and using its exact solution, the expected cycles saved per invocation, as the quantity that every allocation decision is based on. The remaining parts adapt known ideas, and we credit them as such: the chordality of SSA interference graphs and maximum cardinality search [14], [37], coalescing of φ-related values (here as coalitions that bid together), a capacity-k sealed-bid auction that is solved by sorting, best-response dynamics [39], and edge profiling with static fallbacks [27]. A secondary contribution is the evaluation vehicle itself: a complete, tested compiler and web interface in which every allocator shares the same front end, interference graph and code generator, so that differences between allocators are attributable to the allocation decision alone.

Research questions. The evaluation answers four questions:

RQ1 Does profiling help? Is PGARA with measured probabilities better than the same engine driven by static priors?

RQ2 Does the MDP and auction formulation help beyond profiling? Is PGARA better than simply weighting ac-

cesses by profiled block frequency, and better than graph colouring with profile-weighted spill costs?

RQ3 How robust are the results to the register budget (up to $k = 32$), to independently written programs, to several test inputs, to the quality of the training profile and to the model constants?

RQ4 What does it cost at compile time, including the profiling run, and how does that cost grow with function size?

Section IV-J answers each question explicitly.

The remainder of the paper is organised as follows. Section II reviews fifteen recent works, and the classical work on frequency-based spill costs, and the gaps they leave. Section III formalises the language, the front end, the intermediate representations and the optimiser, then develops the MDP, the gain, the coalitions and the auction, with the algorithms and their properties. Section IV describes the toolchain, the parameters and the results on five metrics against six baselines, followed by the ablations, robustness studies and answers to the research questions. Section V concludes, and the team's contributions are listed after it.

II. RELATED WORKS

We review fifteen works from 2018–2025, grouped by theme, followed by the older line of work on frequency-based spill costs that is closest to PGARA. For each we summarise the proposed solution, its methodology and tools, the reported effectiveness and the research gap relative to PGARA.

A. COMBINATORIAL AND SURVEY WORK ON ALLOCATION

Castañeda Lozano et al. [16] propose Unison, which formulates global register allocation (including spilling, coalescing and live-range splitting) together with instruction scheduling as a single constraint model. Programs are first transformed into linear SSA, the model is solved with a constraint programming solver using problem-specific decomposition, and it is integrated with LLVM for Hexagon, ARM and MIPS. Unison produces provably optimal or near-optimal code for functions of up to about a thousand instructions and improves on LLVM's heuristics for a measurable fraction of them. *Gap:* the objective is a static cost estimate, and solve times grow quickly with function size.

Castañeda Lozano and Schulte [17] survey combinatorial approaches to register allocation and instruction scheduling. They classify integer programming, constraint programming, PBQP and related models by the subproblems covered, by scalability and by the quality of the code produced, and compare reported evaluations across targets. The survey shows that combinatorial methods reach optimality on medium-sized functions but are seldom used in production. *Gap:* almost all surveyed models assume static frequency estimates rather than measured, stochastic behaviour.

B. LEARNING-BASED ALLOCATION AND PROGRAM REPRESENTATION

VenkataKeerthy et al. [19] present RL4ReAl, a reinforcement-learning register allocator. The interference graph is embedded with IR2Vec features and a hierarchy of agents learns to pick nodes to colour, spill or split, trained with policy-gradient methods inside an LLVM-based gRPC environment for x86 and AArch64. The learned allocator matches or improves on LLVM's greedy allocator on several benchmark suites. *Gap:* training is costly and individual decisions are not interpretable as expected savings.

Trofin et al. [18] introduce MLGO, a framework for integrating trained policies in LLVM. Policies for inlining-for-size and, later, register-allocation eviction are trained offline with reinforcement learning and evolution strategies on corpora of production code and shipped as compiled TensorFlow models. Inlining-for-size achieved several percent size reduction on large binaries. *Gap:* policies generalise from a corpus, not from the profile of the program being compiled.

Cummins et al. [21] present CompilerGym, a set of environments that expose compiler optimisation problems through the OpenAI Gym interface. It wraps LLVM phase ordering, GCC flag tuning and loop-nest generation behind a client-server architecture, with datasets of millions of programs and multiple observation spaces. It made reinforcement-learning experiments on compilers reproducible and fast. *Gap:* it targets pass-level decisions, not per-value register decisions.

Cummins et al. [22] propose ProGraML, a graph representation of programs that combines control, data and call flow, processed by gated graph neural networks. The representation is built from LLVM and XLA IR and evaluated on data-flow analyses and heuristics such as device mapping and thread coarsening. It outperformed sequence-based representations on those tasks. *Gap:* it learns to approximate analyses that PGARA computes exactly.

VenkataKeerthy et al. [20] introduce IR2Vec, distributed embeddings of LLVM IR derived from a learned seed-embedding vocabulary and flow analysis. Embeddings are computed at instruction, function and program level without heavy training, and used for heterogeneous device mapping and thread-coarsening prediction with competitive accuracy. *Gap:* embeddings capture syntax and flow, not execution frequencies.

Jain et al. [40] propose POSET-RL, which orders optimisation passes with reinforcement learning. A partially ordered set of pass sub-sequences derived from LLVM's O-levels defines the action space, and a deep Q-network trained on IR2Vec states optimises size and execution time. It reduced code size and runtime relative to the standard pipeline on SPEC and MiBench. *Gap:* it operates above the allocator and treats it as a black box.

Haj-Ali et al. [41] present NeuroVectorizer, which learns vectorisation and interleaving factors with deep reinforcement learning. Loops are embedded with code2vec and a policy chooses pragmas that Clang/LLVM applies, with rewards

measured by execution time. It outperformed the baseline cost model on a range of loop kernels. *Gap:* measured rewards drive training, but the decision is not grounded in a model of the program's behaviour.

C. SURVEYS OF MACHINE LEARNING IN COMPILERS

Wang and O'Boyle [2] survey machine learning in compiler optimisation. They organise the field by feature engineering, learning models and deployment, covering search-based tuning, predictive modelling and deep learning across many published systems, and show consistent gains over hand-written heuristics. *Gap:* register allocation is barely represented, and the role of measured profiles inside heuristics is left open.

Ashouri et al. [3] survey compiler autotuning with machine learning, with an emphasis on optimisation selection and phase ordering. They classify works by learning model, features and prediction type and summarise the tools used, from Milepost-GCC to LLVM-based frameworks, reporting the speed-ups achieved. *Gap:* the focus is on choosing passes, not on resource assignment inside a pass.

D. PROFILE-GUIDED OPTIMISATION

Panchenko et al. [23] present BOLT, a post-link optimiser that uses sampled hardware profiles. It rewrites binaries using LLVM's MC infrastructure and reorders basic blocks and functions according to Linux `perf` profiles collected in production. It gave substantial speed-ups on large data-centre applications already built with feedback-directed optimisation. *Gap:* it uses profiles for layout only; register decisions are fixed before it runs.

Shen et al. [24] present Propeller, a profile-guided relinking optimiser for warehouse-scale applications. It integrates profile-driven basic-block layout into the build through distributed, incremental relinking with the LLVM toolchain, avoiding BOLT's monolithic binary rewrite. It achieved comparable speed-ups with much lower memory and build-time cost. *Gap:* as with BOLT, the relinker does not revisit register decisions; inside the compiler the profile reaches allocation only as block-frequency weights on spill costs (Section II-F).

Rotem and Cummins [25] predict branch probabilities statically so that profile-guided optimisation can run without profiles. A gradient-boosted model is trained on features of branches in a large corpus of profiled code and its predictions are fed into LLVM's PGO pipeline. A substantial share of the benefit of real profiles was recovered. *Gap:* predicted probabilities feed existing static heuristics rather than an allocation model that uses them directly.

E. INFRASTRUCTURE AND LARGE LANGUAGE MODELS

Lattner et al. [1] introduce MLIR, a multi-level compiler infrastructure with extensible dialects, regions and SSA-based IR that is shared across domains. It is implemented in C++ within the LLVM project and used to build compilers for machine learning, hardware design and more. It reduced the cost of building domain-specific compilers. *Gap:* MLIR lowers to LLVM's allocators and does not change how registers are assigned.

Cummins et al. [42] present LLM Compiler, a family of foundation models for compiler optimisation. Code models are further trained on a large corpus of LLVM IR and assembly to emulate the compiler, then fine-tuned to choose optimisation pass lists for code size and to disassemble machine code back into IR. The models recover a substantial share of the code-size savings of an autotuning search without any additional compilations, and they are released for further research. *Gap:* their outputs are not guaranteed correct and give no account of the decisions made.

F. FREQUENCY-BASED SPILL COSTS AND PROFILE-GUIDED ALLOCATION

This line of work is the closest to PGARA's central idea, and we review it separately because it predates the 2018–2025 window. *Wall* [28] compared real and estimated profiles as inputs to optimisations including link-time register allocation, and found that estimated profiles recover only part of the benefit of measured ones. *Callahan and Koblenz* [29] colour a hierarchy of tiles that follows the program structure and use execution-frequency estimates to place spill code in infrequently executed tiles. *Briggs et al.* [43] and *George and Appel* [44] established optimistic colouring and conservative and iterated coalescing, with spill costs weighted by estimated frequency, which is the form of graph-colouring allocator we use as a baseline. *Lueh et al.* [45] allocate regions in order of their execution frequency and fuse them, and *Koes and Goldstein* [30] formulate global allocation as a multi-commodity network-flow problem whose costs are weighted by expected execution frequency and refine it progressively. In LLVM, the greedy allocator computes the spill weight of a live range from the block frequencies of its uses and definitions [11], and those frequencies are derived from profile data when it is supplied [12]. *Gap:* in all of these, the profile scales an access count; none states, per value, the expected cycles that residency saves under a model of spills, reloads, calls and forwarding, and none makes loop-carried values bid as a coalition. Section IV-G tests whether this difference matters by comparing PGARA with a profile-weighted priority allocator and with Chaitin–Briggs colouring using profile-weighted spill costs.

G. SUMMARY OF GAPS

Across these works, measured execution frequencies drive code layout, and inside the allocator they enter only as weights on spill costs; learned allocators require heavy training and are opaque; and combinatorial allocators optimise static cost models. PGARA addresses the intersection: an allocator whose objective is the expected saving under measured probabilities, solved exactly and cheaply, with every decision explainable as a bid.

III. PROPOSED WORK

A. PROBLEM FORMULATION

Let a function in SSA form have values V, a control-flow graph (CFG) $C = (B, E, b_0)$ and a register file of k registers. An allocation is a map $\rho : V \to \{0, \ldots, k-1\} \cup \{\perp\}$ in which $\rho(v) = \perp$ means that v lives in its stack slot; it is feasible when no two interfering values share a register. Executing the program on an input induces a random walk over C, and the cost of the allocation is the expected number of cycles spent on loads, stores and copies caused by values without a register. We seek

$$\rho^{\star} = \arg\max_{\rho \text{ feasible}} \sum_{v:\, \rho(v)=\perp} G(v), \tag{1}$$

where $G(v)$ is the expected cycles saved by giving v a register, computed under transition probabilities measured by profiling.

Two features make (1) tractable without colouring. On strict SSA form the interference graph is chordal [14], so the values competing at any program point form a clique that an MCS order exposes one at a time. And the gain of a single value can be computed exactly by stochastic dynamic programming over its live range, because the residency of one value is a small MDP whose only uncertainty is the branch behaviour of the CFG. Both observations are developed below, after the front end and middle end that produce the SSA program.

B. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture. The front end turns MiniC source text into a type-checked abstract syntax tree (AST) and symbol table; the middle end lowers it to three-address code (TAC), builds the CFG, constructs SSA and optimises it; a reference interpreter executes the optimised SSA on a training input and records edge counts; the PGARA engine converts those counts into gains, coalitions and an auctioned allocation; and the back end leaves SSA, emits PG-RISC code and runs it on a cycle-counting simulator on a separate test input.

Every stage reports its result into a single JSON document, which the web interface renders as token streams, AST, symbol tables, TAC, SSA, CFG, interference graph, auction rounds and target code (dashed arrows in Fig. 1). Three baseline allocators share the same pipeline and code generator, so differences in the measured cycles are attributable to allocation decisions alone.

C. FRONT END: LANGUAGE, GRAMMAR AND PARSING

Lexical analysis

MiniC has `int` scalars and one-dimensional arrays, array parameters passed by reference, functions, `if`/`else`, `while`, `for`, `break`, `continue`, `print` and `input()`, the arithmetic, relational and short-circuit logical operators, and the compound forms `+=`, `-=`, `*=`, `++`, `--`. The lexer is a deterministic finite automaton applied under the maximal-munch rule: from position i it returns the longest prefix w of the remaining input with $\delta^*(q_0, w) \in F$, so `<=` is one token and `a+++b` is `a ++ + b`. Keywords are recognised as identifiers and reclassified by table lookup. An invalid character is reported with its line and column and skipped (panic mode), so later errors are still found in the same run.

Context-free grammar

The syntax is the context-free grammar

$$G = (N, \Sigma, P, S), \quad |N| = 41, \ |P| = 94, \ S = \mathit{Program}, \tag{2}$$

written directly in LL(1) form. Left recursion is removed by the standard tail transformation and precedence is encoded by one nonterminal per level, for example

$$\begin{aligned} \mathit{Add} &\to \mathit{Mul}\ \mathit{AddRest} \\ \mathit{AddRest} &\to + \ \mathit{Mul}\ \mathit{AddRest} \mid - \ \mathit{Mul}\ \mathit{AddRest} \mid \varepsilon \\ \mathit{Stmt} &\to \texttt{if}\ (\ \mathit{Expr}\ )\ \mathit{Stmt}\ \mathit{ElsePart} \\ &\quad \mid \texttt{while}\ (\ \mathit{Expr}\ )\ \mathit{Stmt} \mid \cdots \\ \mathit{ElsePart} &\to \texttt{else}\ \mathit{Stmt} \mid \varepsilon. \end{aligned}$$

The precedence chain, from loosest to tightest binding, is *Or*, *And*, *Eq*, *Rel*, *Add*, *Mul*, *Unary*, *Primary*, giving the usual C precedence and left associativity.

FIRST, FOLLOW and the LL(1) property

For $\alpha \in (N \cup \Sigma)^*$ and $A \in N$,

$$\begin{aligned} \mathrm{FIRST}(\alpha) &= \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\}, \\ \mathrm{FOLLOW}(A) &= \{a \in \Sigma \cup \{\$\} \mid S\$ \Rightarrow^* \beta A a \gamma\}. \end{aligned} \tag{3}$$

Both are computed as least fixpoints of the usual inclusion constraints. G is LL(1) iff for every pair of alternatives $A \to \alpha \mid \beta$

$$\mathrm{FIRST}\ \alpha\, \mathrm{FOLLOW}(A) \cap \mathrm{FIRST}\ \beta\, \mathrm{FOLLOW}(A) = \varnothing. \tag{4}$$

The toolchain computes (3), builds the predictive table and checks (4) mechanically. The single conflict is the cell (*ElsePart*, `else`): `else` is in $\mathrm{FIRST}(\texttt{else}\ \mathit{Stmt})$ and, because *ElsePart* can end a *Stmt*, also in $\mathrm{FOLLOW}(\mathit{ElsePart})$. This is the dangling-else ambiguity, resolved by preferring $\mathit{ElsePart} \to \texttt{else}\ \mathit{Stmt}$ so that an `else` binds to the nearest unmatched `if`. Every other cell holds at most one production, so G is LL(1) modulo this resolution.

Relation to LR parsing

Every LL(1) grammar is also LR(1) [46], [47], so an LR(1) or LALR parser generator would accept G unchanged apart from the same dangling-else conflict, which appears there as a shift/reduce conflict on `else` and is conventionally resolved in favour of shift, binding `else` the same way. We chose top-down predictive parsing because a recursive-descent parser is a direct transcription of the grammar (one procedure per nonterminal), errors can be reported in terms of the construct being parsed, and the GUI can show the predictive table, which students can check against (3)–(4). The

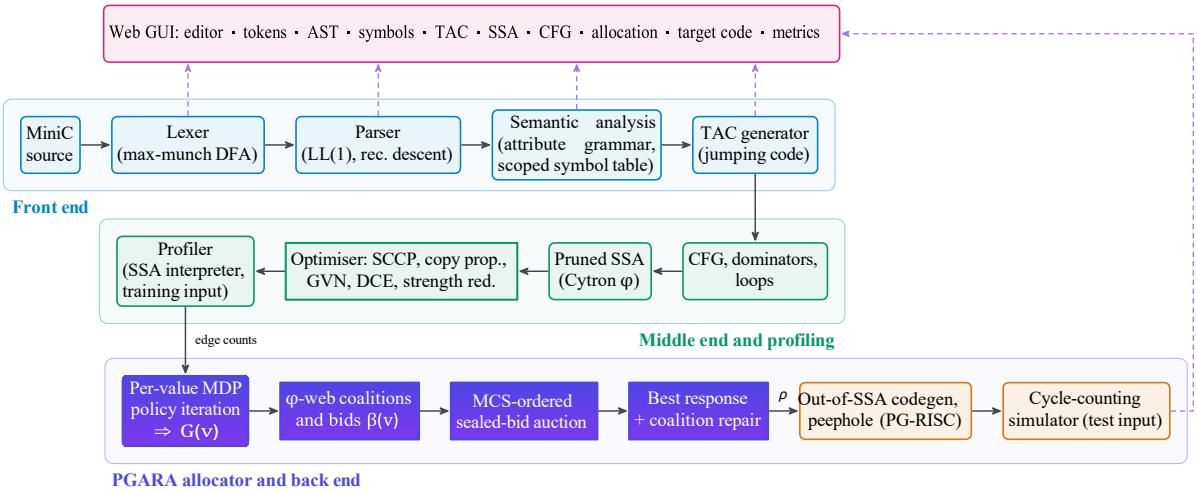


FIGURE 1. Architecture of the PGARA compiler: front end (blue), middle end and profiling (green), the PGARA allocation engine (indigo) and back end (amber). Dashed violet arrows show phase reports consumed by the web interface (pink).

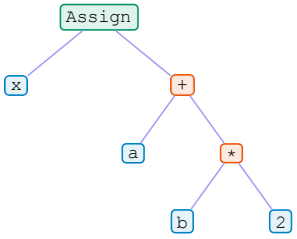
parser recovers from syntax errors in panic mode, discarding tokens until one in the FOLLOW set of the current construct (for statements, `;` or `}`) is found.

D. SEMANTIC ANALYSIS AS AN ATTRIBUTE GRAMMAR

Semantic analysis is specified as an L-attributed grammar [48] and implemented as a traversal of the AST. Each node carries an inherited environment Γ (a stack of scopes, each a hash map from names to symbols) and a synthesised type $\tau \in \{\texttt{int}, \texttt{int[]}, \texttt{void}\}$. Typical rules are

$$
\begin{aligned}
&\mathit{Add} \rightarrow \mathit{Mul}\ \mathit{AddRest}: \\
&\quad \mathit{AddRest.in} = \mathit{Mul.node},\ \ \mathit{Add.node} = \mathit{AddRest.out} \\
&\mathit{AddRest} \rightarrow + \ \mathit{Mul}\ \mathit{AddRest}_1: \\
&\quad \mathit{AddRest}_1\mathit{.in} = \mathrm{Bin}(+, \mathit{AddRest.in}, \mathit{Mul.node}), \\
&\quad \mathit{AddRest.out} = \mathit{AddRest}_1\mathit{.out} \\
&\mathit{AddRest} \rightarrow \varepsilon: \quad \mathit{AddRest.out} = \mathit{AddRest.in} \\
&E \rightarrow E_1\ \mathit{op}\ E_2: \quad E.\tau = \texttt{int} \text{ iff } E_1.\tau = E_2.\tau = \texttt{int} \\
&\mathit{Primary} \rightarrow \texttt{ID}: \quad \mathit{Primary}.\tau = \Gamma(\texttt{ID}).\tau \text{ (error if unbound)}
\end{aligned}
\tag{5}
$$

The inherited attribute *in* restores left associativity that the LL(1) transformation removed from the syntax. Fig. 2 shows the flow of attributes on a small statement. Declarations extend Γ in the innermost scope and reject redeclaration in the same scope; uses resolve outward through enclosing scopes. Functions are collected in a first pass, so calls may precede definitions. The analyser checks arity and argument kinds, array indexing and assignment, use of `void` results, `break`/`continue` outside loops and missing returns. Every symbol receives a unique IR name: its own name, or *x#n* when that name is already used in the function, so shadowed variables never collide in later phases.



Attribute	Flow on `x = a + b * 2;`
Γ (inherited)	passed down to every leaf; resolves `x`, `a`, `b`
τ (synthesised)	leaves `int`; `*`, `+` give `int`; Assign checks
in (inherited)	builds `a + (b*2)` left-associatively
unique name	`x`, `a`, `b`; a second `i` becomes `i#1`

FIGURE 2. Attributed AST of a simple assignment, coloured as in the GUI: statements green, expressions orange, leaves blue. The environment flows down, types flow up, and each identifier is bound to a unique IR name.

E. THREE-ADDRESS CODE AND CONTROL FLOW

The AST is lowered by a syntax-directed translation into three-address code whose instructions have the forms $x = y\ \mathit{op}\ z$, $x = \mathit{op}\ y$, $x = y$, $x = a[i]$, $a[i] = y$, `goto` L, `if` x `goto` L_t `else` L_f, `param`, `call`, `return`, `print` and `input`. Conditions are translated as jumping code [8]: with true and false targets t, f,

$$
\begin{aligned}
\mathbf{J}\mathbf{J}B_1 \ \&\&\ B_2)(t,f) &= \mathbf{J}\mathbf{J}B_1)(L,f)\ ;\ L\!:\mathbf{J}\mathbf{J}B_2)(t,f), \\
\mathbf{J}\mathbf{J}B_1 \ ||\ B_2)(t,f) &= \mathbf{J}\mathbf{J}B_1)(t,L)\ ;\ L\!:\mathbf{J}\mathbf{J}B_2)(t,f), \\
\mathbf{J}\mathbf{J}\,!\,B)(t,f) &= \mathbf{J}\mathbf{J}B)(f,t),
\end{aligned}
\tag{6}
$$

with L a fresh label, so short-circuit evaluation is exact and no Boolean value is materialised unless it is stored. Basic blocks are maximal sequences with a single entry and a single exit. The CFG is cleaned of unreachable blocks and its critical edges are split, so that copies inserted later on an edge have a block of their own. Dominators are computed by the Cooper–Harvey–Kennedy iterative algorithm over reverse postorder [49], natural loops are identified from back edges

TAC

```
entry: n = input()
       s = 0
       i = 0
for1:  t2 = i < n
       if t2 goto body2
            else endfor4
body2: t3 = i % 3
       t4 = t3 == 0
       if t4 goto and7
            else endif6
and7:  t5 = i > 0
       if t5 goto then5
            else endif6
then5: s = s + i
endif6:
step3: i = i + 1
       goto for1
endfor4:
       print s
       return 0
```

Optimised SSA

```
entry: n = input()
       goto for1
for1:  i.2 = phi(0, i.3)
       s.2 = phi(0, s.4)
       if i.2 < n goto body2
            else endfor4
body2: t3 = i.2 % 3
       if t3 == 0 goto and7
            else endif6
and7:  if i.2 > 0 goto then5
            else endif6
then5: s.3 = s.2 + i.2
endif6:
       s.4 = phi(s.2, s.2, s.3)
       i.3 = i.2 + 1
       goto for1
endfor4:
       print s.2
       return 0
```

FIGURE 3. Compiler output for `for(i=0;i<n;i++) if(i%3==0 && i>0) s+=i;`. Jumping code realises `&&`; after SSA and optimisation, comparisons are fused into branches and constants into φ arguments (split critical-edge blocks omitted).

$n \to h$ with h dom n, and each block gets a loop depth. Fig. 3 shows the TAC and optimised SSA the compiler produces for a small loop.

F. SSA CONSTRUCTION AND OPTIMISATION

Liveness is the least fixpoint of

$$\begin{aligned} \mathit{LiveIn}(b) &= \mathit{use}(b) \cup \big(\mathit{LiveOut}(b) \setminus \mathit{def}(b)\big), \\ \mathit{LiveOut}(b) &= \bigcup_{s \in \mathit{succ}(b)} \mathit{LiveIn}(s), \end{aligned} \tag{7}$$

solved by a worklist over reverse postorder, with φ-uses attributed to the corresponding predecessor edge. SSA is built by the method of Cytron et al. [5]: for each variable x with definition blocks D_x, a φ is placed at every block of the iterated dominance frontier $DF^{+}(D_x)$ at which x is live on entry (pruned SSA), and variables are renamed by a walk of the dominator tree. A read with no reaching definition becomes the constant 0, which matches MiniC's zero-initialisation of locals.

The optimiser iterates, to a fixpoint or ten rounds, sparse conditional constant propagation over the lattice $\top \sqsupset c \sqsupset \bot$ with executable-edge tracking (SCCP) [50], copy propagation, algebraic simplification and strength reduction (for example $x \cdot 2^m \Rightarrow x \ll m$, $x + 0 \Rightarrow x$), global value numbering scoped by the dominator tree, dead-code elimination over SSA def-use chains and the merging of straight-line blocks. It then splits critical edges again and fuses each comparison with the branch that consumes it into a compare-branch instruction. Semantic preservation is checked on every benchmark by executing the SSA program before and after optimisation and comparing outputs.

G. PROFILING AND TRANSITION PROBABILITIES

The optimised SSA program is executed by a reference interpreter with bounds-checked memory on a training input. The

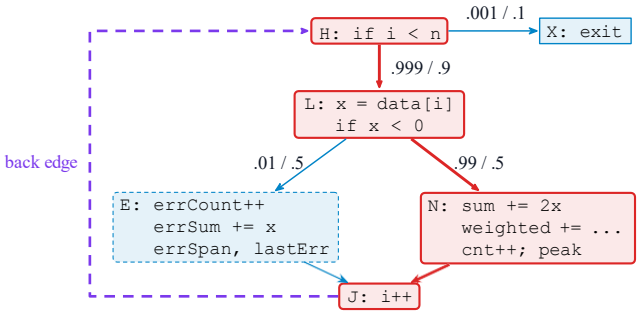


FIGURE 4. Inner loop of the rare-branch benchmark. Edge labels give P_{prof}/P_{stat}. Red marks the hot path found by profiling and blue the rarely executed blocks: the error block E runs in 1% of iterations, whereas the static prior assumes 50%.

interpreter counts every CFG edge $n(b, s)$, and the profiled transition matrix is

$$P_{\mathrm{prof}}(b, s) = \frac{n(b, s)}{\sum_{s' \in \mathit{succ}(b)} n(b, s')}, \tag{8}$$

falling back to the static prior for blocks never reached in training. The Static-Prior variant uses the classical loop heuristic [27] instead: if exactly one successor leaves a loop containing b,

$$P_{\mathrm{stat}}(b, s) = \begin{cases} 0.1 & s \text{ exits the loop}, \\ 0.9 & \text{otherwise}, \end{cases} \tag{9}$$

and otherwise $P_{\mathrm{stat}}(b, s) = 1/|\mathit{succ}(b)|$. Fig. 4 contrasts the two on the inner loop of the rare-branch benchmark, where an error path is taken in about 1% of iterations but the static prior sees a fair coin.

H. THE RESIDENCY MDP OF A VALUE

Definition 1 (Residency MDP): For an SSA value v with live range $L_v \subseteq B$, the residency MDP has states (b, s) with $b \in L_v$ and $s \in \{\mathsf{R}, \mathsf{M}\}$ (control is in block b and v is in a register or in its slot). In state R the actions are stay and spill (cost c_{st}); in state M they are stay and reload (cost c_{ld}). Control moves from b to $b' \in L_v$ with probability $P(b, b')$ when $v \in \mathit{LiveIn}(b')$; leaving the live range is absorbing with value 0.

Let $u_v(b)$, $d_v(b)$ and $f_v(b)$ be the numbers of uses, definitions and forwardable uses (a use immediately after the definition, which the peephole pass turns into a register move when v is spilled) of v in b, and $c_v(b)$ the number of calls in b across which v is live. The per-block rewards are

$$\begin{aligned} r_{\mathsf{R}}(b) &= \sigma\big(u_v(b) + d_v(b) + f_v(b)\big) - c_v(b)\,(c_{\mathrm{st}} + c_{\mathrm{ld}}), \\ r_{\mathsf{M}}(b) &= -\,u_v(b)\,c_{\mathrm{ld}} + d_v(b)\,c_{\mathrm{st}} + f_v(b)\,c_{\mathrm{mv}}, \end{aligned} \tag{10}$$

with σ a small bonus for a register access, c_{mv} the cost of a forwarded move and an extra $-c_{\mathrm{ld}}$ in the definition block of a parameter, which arrives in its slot. The constants are taken from the simulator's latency table rather than tuned: c_{ld} and c_{st} are the load and store latencies, $c_{\mathrm{mv}} = 1$ is the latency of the register move that replaces a forwarded load, and $\sigma = 1$, one cycle per access, is the unit of the latency

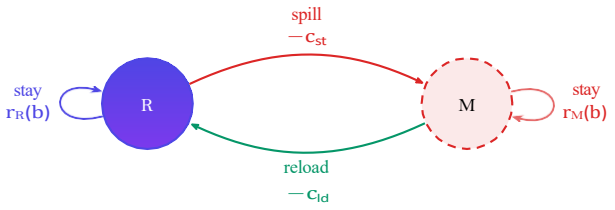


FIGURE 5. The two-state residency MDP replicated over each block b of a value's live range. In either state control moves to b′ with probability P (b, b′); leaving L_v is absorbing with value 0. Rewards follow (10).

table. σ is not a physical cost: it keeps the gain of an accessed value strictly positive and ranks values with equal spill traffic by how often they are accessed. Section IV-H shows that the results change by at most 0.4% when σ and c_{mv} are varied over {0, 0.5, 1, 2} and {0, 1, 2}. The optimal values satisfy the Bellman equations [32], [51]

$$\begin{aligned} V(b,\mathsf{R}) &= r_{\mathsf{R}}(b) + \max\{\bar{V}_{\mathsf{R}}(b),\ -c_{st} + \bar{V}_{\mathsf{M}}(b)\}, \\ V(b,\mathsf{M}) &= r_{\mathsf{M}}(b) + \max\{\bar{V}_{\mathsf{M}}(b),\ -c_{ld} + \bar{V}_{\mathsf{R}}(b)\}, \\ \bar{V}_s(b) &= \sum_{b' \in L_v} P(b,b')\, V(b',s), \end{aligned} \tag{11}$$

Fig. 5 draws the two states. Because a live range is left with probability one under every policy of a terminating program, the chain restricted to L_v is transient and (11) is solved undiscounted, as a total-reward MDP. For a fixed policy π, policy evaluation is the sparse linear system

$$I - P_\pi\ V_\pi = r_\pi, \tag{12}$$

where P_π is substochastic and transient, so $I - P_\pi$ is a non-singular M-matrix and Gaussian elimination without pivoting is stable. Howard's policy iteration [34] alternates (12) with greedy improvement and terminates in a finite number of rounds with the exact optimum; we keep value iteration as an independent cross-check in the test suite. If the restricted chain were ever not transient, the solver would fall back to a discount factor $\gamma = 0.999$.

Complexity of Algorithm 1. The block frequencies (13) need one sparse solve over the $|B|$ blocks, $O(|B|^3)$ in the worst case for Gaussian elimination. For a value v, every policy-iteration round solves (12), a system of $2|L_v|$ unknowns, which costs $O(|L_v|^3)$ in the worst case and $O(|L_v|\ w^2)$ when the system, ordered by reverse postorder, has bandwidth w; the improvement step is $O(|L_v|)$. With R_v rounds and the two forced-policy evaluations, the gains cost $O\left(|B|^3 + \sum_v (R_v + 2)\,|L_v|^3\right)$ in the worst case. Howard's method needs few rounds in practice (1.3 on average and at most 7 in our suites), and coalitions are a union-find pass over the φ-edges. The MCS order is $O(|V| + |A|)$, each auction round sorts at most $k+1$ bids, and binding is $O(|V|\ k + |A|)$; best response makes at most $2|V|$ moves, each checked in $O(|A|)$, so the auction and equilibrium steps are dominated by the gains. Section IV-F measures how the allocation time grows with function size and live-range length.

I. EXPECTED GAIN OF A VALUE

Neither outcome of the auction leaves the value free to choose its policy. A loser is forced to stay in M throughout; let V_{mem} be the value of that policy, obtained from (12) with $\pi \equiv$ stay. A winner is forced to stay in R throughout, because the code generator does not split live ranges (Section III-M); let V_{reg} be the value of that policy, again from (12) with $\pi \equiv$ stay. The per-entry gain is the difference between the two policies that can actually be realised, $g(v) = V_{\text{reg}}(def\,(v), \mathsf{R}) - V_{\text{mem}}(def\,(v), \mathsf{M})$. The optimal value V^* of (11) is still computed: it bounds the realised gain from above, since always staying in R is one admissible policy, it shows in the GUI where a value would rather be spilled or reloaded, and it is the quantity a live-range-splitting code generator would use. The first draft of this paper bid $V^*(def\,(v), \mathsf{R}) - V_{\text{mem}}$, which values a policy that is never carried out; Section IV-H shows that on our suites the two bids differ by 0.05% in total and give identical cycle counts in 48 of 51 configurations. The live range is entered once for each execution of the definition block, whose expected count per invocation ϕ solves the Markov-chain flow equations

$$\phi = e_{b_0} + P^{\top}\phi, \tag{13}$$

so the expected register-residency gain of v per invocation is

$$G(v) = \phi\ def\,(v)\ \cdot\ V_{\text{reg}}\ def\,(v), \mathsf{R}\ -V_{\text{mem}}\ def\,(v), \mathsf{M}\ . \tag{14}$$

Measuring against V_{mem} rather than against $V^*(\cdot, \mathsf{M})$ matters: an optimal policy that starts in M may reload at once, which would cap every gain at roughly c_{ld} and hide the difference between hot and cold values. $G(v)$ is the quantity a compiler engineer would ask for (expected cycles saved), and the GUI shows it for every value. In the rare-branch example of Fig. 4, the profiled MDP rates the hot-path accumulators `cnt.3` and `weighted.3` at $G = 20{,}300$ and the error-path counters `errSum.3` and `errCount.3` at $G = 180$, a factor of 113. Under the static prior (9) all four receive the same gain, 202.5, because the two arms of the branch look alike; Static-Prior consequently keeps the error-path value `lastErr` in a register and spills `cnt` and `weighted`, whereas PGARA does the opposite.

J. COALITIONS ON φ-WEBS

A loop-carried variable is a chain of SSA values joined by φ-functions. If they are allocated independently, the auction may grant a register to one version and not the next, and every traversal of the back edge then pays a copy through memory. We therefore form coalitions by union-find over φ edges, merging the argument a and result p of a φ in block h along edge (b, h) only when the edge is hot:

$$\phi(b)\,P(b,h) \ \geq\ \vartheta \cdot \max\{\phi(def\,(a)),\ \phi(h)\}, \qquad \vartheta = 0.75. \tag{15}$$

A loop-carried variable thus forms one coalition around its back edge, while a copy made once on loop entry, or on a rare path, does not drag a cold value into a hot coalition. The

threshold ϑ was chosen on a separate tuning suite of five programs that is not used in any reported result (Section IV-B). Every member of coalition C bids the coalition's total gain,

$$\beta(v) = \sum_{w \in C(v)} G(w), \tag{16}$$

so the coalition wins or loses as a unit. This is the transferable-utility view of cooperative game theory [35]: the coalition's value is what its members achieve together, and (16) lets members with small individual gains (for example the version of `i` after the increment, $G = 12{,}288$) inherit the bid of the whole web ($\beta = 94{,}223$).

K. SEALED-BID AUCTION ON THE CHORDAL INTERFERENCE GRAPH

The interference graph $\mathbf{I} = (V, A)$ joins two values when one is live at the definition of the other. On strict SSA $\mathbf{I}$ is chordal [14]; we compute a maximum cardinality search order σ [37] and verify on every function that its reverse is a perfect elimination ordering (PEO), which held for all functions in our suite. In a chordal graph visited in MCS order, the already-visited neighbours of each vertex form a clique, so when v arrives its already-visited neighbours together with v are a clique Q_v, a set of values that are all simultaneously live.

The auction (Algorithm 2) visits values in order σ. If v has fewer than k already-winning neighbours, it wins outright. Otherwise Q_v exceeds the register budget and a sealed-bid round awards the k registers of that clique to the k highest bids:

$$W(Q_v) = \arg\max_{S \subseteq Q_v,\ |S| \le k} \sum_{w \in S} \beta(w), \tag{17}$$

which for a single capacity constraint is solved exactly by sorting. Ties are broken by σ, so the result is deterministic. Losers are spilled to slots that are shared by every member of a φ-web, so that spilled φ copies become no-ops. Registers are then bound along σ, choosing for each winner a register not held by any winning neighbour and preferring one already held by a member of its web, so the φ copy disappears. Fig. 6 shows a real auction round from the rare-branch benchmark at $k = 4$.

L. BEST-RESPONSE DYNAMICS AND EQUILIBRIUM

The auction decides each clique locally, so a loser can end up with a register that none of its neighbours actually uses (for example because a rival that beat it later lost elsewhere). We model the result as a game whose players are coalitions, whose strategies are a register for each member or memory, and whose payoff is β when all members hold registers and 0 otherwise, subject to the non-interference constraint. A spilled coalition C has a profitable unilateral deviation if

$$\beta(C) > 0 \wedge \forall v \in C : \{0, \ldots, k-1\} \setminus \rho\ \mathrm{adj}(v) \neq \varnothing. \tag{18}$$

Best-response dynamics (Algorithm 3) let deviating coalitions take free registers, highest bid first, until none can.

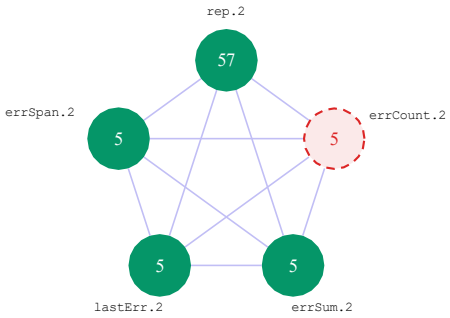


FIGURE 6. An auction round at k = 4 in the rare-branch benchmark: when `errCount.2` arrives in MCS order its clique has five members. Node labels are bids β; the four highest win (green), and the tie at 5 is broken by MCS order, so `errCount.2` is spilled (red, dashed). The GUI uses the same colours for auction winners and losers.

Algorithm 1 Profile-driven gains and coalition bids

Require: SSA function with CFG $\mathbf{C}$, edge counts n (or none)
Ensure: gains G, bids β, coalitions C

1: $P \leftarrow$ (8) if n given, else (9)
2: $\phi \leftarrow$ solve $(I - P^{\top})\phi = e_{b_0}$ (13)
3: **for all** $v \in V$ **do**
4: build rewards r_R, r_M on L_v (10)
5: $\pi \leftarrow$ *stay* in every state
6: **repeat**
7: $V_\pi \leftarrow$ solve $(I - P_\pi)V_\pi = r_\pi$ (12)
8: $\pi' \leftarrow$ greedy in (11) w.r.t. V_π
9: **if** $\pi' = \pi$ **then break**; $\pi \leftarrow \pi'$
10: **until** false
11: $V^* \leftarrow V_\pi$ {optimal; kept for display and splitting}
12: $V_{\mathrm{reg}}, V_{\mathrm{mem}} \leftarrow$ evaluate $\pi \equiv$ *stay* from R and from M
13: $g \leftarrow V_{\mathrm{reg}}(def(v), \mathsf{R}) - V_{\mathrm{mem}}(def(v), \mathsf{M})$
14: $G(v) \leftarrow \phi(def(v)) \cdot g$ (14)
15: **end for**
16: $C \leftarrow$ union-find over φ edges satisfying (15)
17: $\beta(v) \leftarrow \sum_{w \in C(v)} G(w)$ for all v (16)
18: **return** G, β, C

Splitting a coalition between registers and memory costs a copy through memory on each hot φ edge that joins a winner and a loser. When that expected penalty is at least the winners' own gain,

$$\sum_{\substack{(b,h)\ \varphi\text{-edge} \\ \text{split}}} \phi(b)\, P(b,h)\, c_{(b,h)} \ge \sum_{w \in C,\ \rho(w) = \perp} G(w), \tag{19}$$

with $c_{(b,h)}$ equal to c_{ld} or c_{st} depending on the direction of the copy, the winners concede and the whole coalition leaves the game: it is added to an exclusion set X that every later best-response pass skips (Algorithm 3, lines 1, 6 and 13), so a conceded coalition can never take registers again. Best response then runs once more, over the coalitions not in X, to reuse the freed registers.

Proposition 1: Algorithm 3, including the repair step, terminates after at most $2|V|$ deviations and $|V|$ concessions, and its result is a pure Nash equilibrium of the game among the

Algorithm 2 MCS-ordered sealed-bid auction and register binding

Require: interference graph I, bids β, k, webs
Ensure: allocation ρ

$\sigma \leftarrow$ MaximumCardinalitySearch(I); assert reverse(σ) is a PEO
$win \leftarrow \emptyset$
for all v in order σ **do**
  $R \leftarrow \{w \in \mathrm{adj}(v) \mid w \in win\}$
  if $|R| < k$ **then**
    $win \leftarrow win \cup \{v\}$
  else
    $Q \leftarrow R \cup \{v\}$ {a clique}
    $S \leftarrow$ top-k of Q by $(\beta, -\sigma)$ (17)
    $win \leftarrow (win \setminus Q) \cup S$
  end if
end for
for all $v \in win$ in order σ **do**
  $F \leftarrow \{0..k-1\} \setminus \{\rho(w) \mid w \in \mathrm{adj}(v)\}$
  $\rho(v) \leftarrow$ a register of F held by v's web, else $\min F$
end for
return ρ (values without a register are spilled)

Algorithm 3 Best-response dynamics with coalition repair

Require: ρ from Algorithm 2, coalitions C, bids β, gains G
Ensure: equilibrium allocation ρ

$X \leftarrow \emptyset$ {coalitions that have left the game}
BestResponse(ρ, X)
for all coalitions C split between registers and memory **do**
  if (19) holds for C **then**
    $\rho(w) \leftarrow \perp$ for every $w \in C$
    $X \leftarrow X \cup \{C\}$
  end if
end for
if $X \neq \emptyset$ **then** BestResponse(ρ, X)
return ρ

procedure BestResponse(ρ, X)
repeat
  $changed \leftarrow$ false
  for all spilled coalitions $C \notin X$ by decreasing β **do**
    if every $v \in C$ has a free register (18) **then**
      assign them, preferring registers of web partners
      $changed \leftarrow$ true
    end if
  end for
until not $changed$

coalitions not in X: no spilled coalition outside X satisfies (18), and no winning coalition can raise its payoff by deviating.

Proof sketch. Let $\Phi(\rho)$ be the number of values that hold a register. In BestResponse, a deviation moves a whole coalition from memory into registers that are free with respect to all of its neighbours, without evicting anyone, so Φ grows strictly and is bounded by $|V|$: the first pass makes at most $|V|$ deviations. The repair step is a single pass over the coalitions; it only removes values, each coalition is demoted at most once, and every demoted coalition enters X, so at most $|V|$ values are conceded. The second pass skips X, so it cannot re-admit a conceded coalition and cannot undo the repair; by the same argument on Φ it makes at most $|V|$ further deviations and ends when (18) fails for every spilled coalition outside X. Coalitions in X have conceded because the copy penalty (19) of a split allocation exceeds their gain, so they are not players of the final game. A winning coalition already receives its maximal payoff $\beta(C)$, and changing to another free register leaves the payoff unchanged, so no player has a strictly profitable deviation. □

We describe the game cautiously. Because two interfering values may not share a register, it is not a congestion game in Rosenthal's sense [38]: a player's payoff depends on which neighbours in the interference graph hold which registers, not on how many players use a resource, which makes it closer to the graph-colouring games studied by Panagopoulou and Spirakis [52]. We do not claim an exact potential function [53] for the general game, in which players could also switch or evict. For the restricted dynamics of Algorithm 3, which only admit deviations from memory into free registers, Φ above is a potential that increases with every move, which is all that the termination argument needs. The equilibrium need not maximise (1) globally, and such games have many equilibria. The auction is what steers the dynamics towards a high-gain equilibrium: best response only fills registers that the auction left free.

M. BACK END: LEAVING SSA AND CODE GENERATION

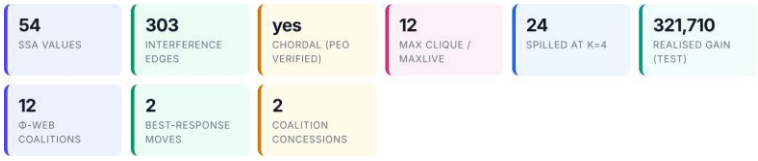
Fig. 8 summarises the allocation flow and its hand-off to the back end. The target, PG-RISC, is a load/store RISC with allocatable registers $r_0, \ldots, r_{k-1}$, three scratch registers t_0–t_2 for spilled operands, a return-value register and a frame pointer. The frame holds parameters, spill slots (one per φ-web), caller-save slots and local arrays. φ-functions are replaced by parallel copies on the (already split) incoming edges, sequentialised with cycles broken through t_2; copies between values that share a register or a slot vanish. Registers live across a call are saved and restored around it. A peephole pass threads jumps, removes jumps to the next instruction and forwards a store to an immediately following load of the same slot, turning the load into a move or deleting it, which is the forwarding that $f_v(b)$ in (10) anticipates.

N. MODULES AND THE TEAM'S CONTRIBUTION

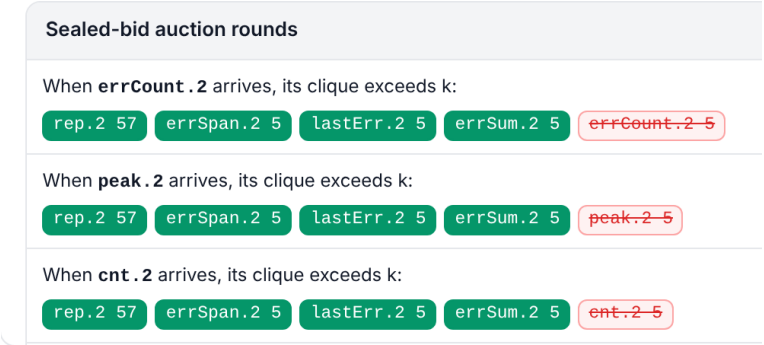
The work divides into modules owned by the team members (the full list is given in the contributions section). The core pipeline module provides the lexer, the grammar and its LL(1) verification, the parser and AST, and TAC generation (Shreyans Chowdry). The semantic module provides the scoped symbol table, the attribute evaluation

Value	G(v)	Bid	Live blocks	PI rounds	Loca
x	101,860.0	101,860.0	4	2	r3
i#1.2	81,935.0	94,223.0	11	2	r1
i#1.3	12,288.0	94,223.0	1	1	r1
cnt.3	20,300.0	32,480.0	8	3	r0
weighted.3	20,300.0	32,480.0	8	3	r2
sum.3	20,300.0	32,480.0	8	3	slo
sum.4	12,180.0	32,480.0	4	1	slo
weighted.4	12,180.0	32,480.0	4	1	r2
cnt.4	12,180.0	32,480.0	4	1	r0
cnt.5	0.0	32,480.0	1	1	r0
weighted.5	0.0	32,480.0	1	1	r2
sum.5	0.0	32,480.0	1	1	slo
t15	20,480.0	20,480.0	1	1	r3
peak.3	20,300.0	20,300.0	9	3	slo
t20	20,300.0	20,300.0	1	1	slo

(a) Gains $G(v)$, coalition bids, live-range size, policy-iteration rounds and location



(b) Summary of the allocation of `main`



(c) The first auction rounds in MCS order: winners in green, losers in red

FIGURE 7. The Allocation view of the PGARA Compiler Studio on the rare-branch benchmark at k = 4 (training input `7 10`, test input `13 10`), cropped to its key panels. The full-window screenshots of every view are provided as supplementary material in the repository (`docs/screenshots`).

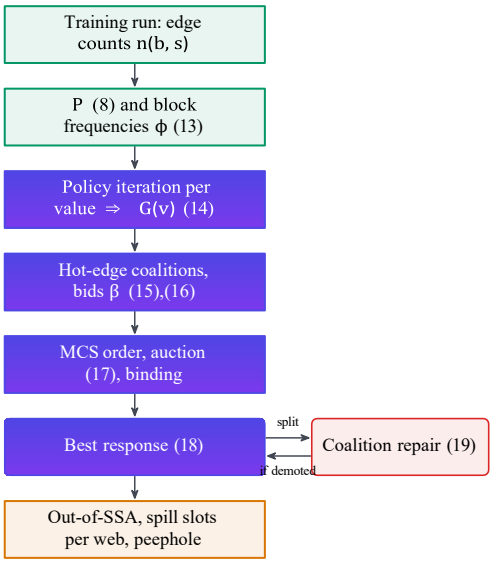


FIGURE 8. Flow of the PGARA allocator, from profile to target code.

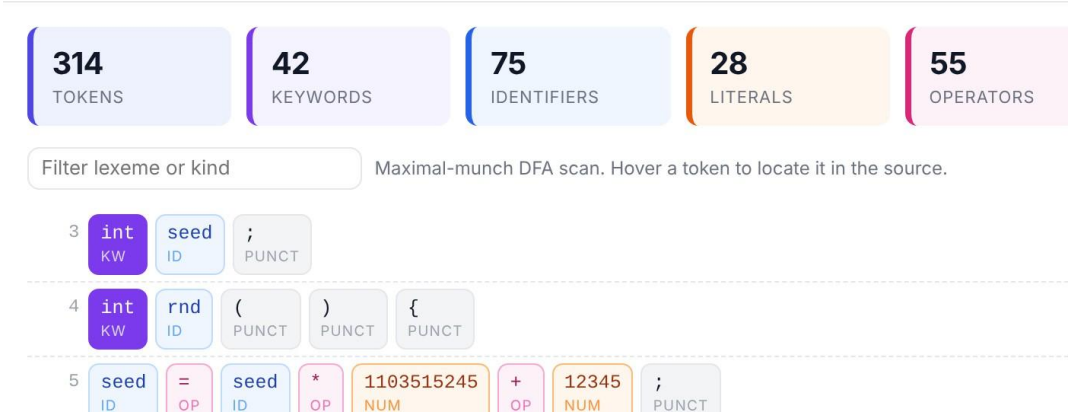
and diagnostics (Samrender Singh Saini). The optimisation and code-generation module provides SSA, the optimiser, the PGARA gain engine and PG-RISC code generation (Karthikeya Jami). The runtime and evaluation module provides the interpreter, the simulator, the test suite and the benchmark harness (Abhishek Kumar). The web GUI and its visualisations are part of the core pipeline module (Shreyans Chowdry), with documentation and interface testing support from Nishanth Kumar. Within the PGARA allocator itself, the work is divided among four members: the profile-derived transition matrices, block frequencies and φ-web coalitions (Shreyans Chowdry); the interference graph, MCS order and sealed-bid auction (Samrender Singh Saini); the residency MDP, policy iteration and gains (Karthikeya Jami); and best response, coalition repair and the baselines (Abhishek Kumar).

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. TOOLS AND SETUP

The compiler is written in Python 3 using only the standard library, so it runs on any machine with a recent Python installation and no package manager [54]. A small HTTP server from the standard library serves the web interface and a JSON compilation endpoint; the interface is plain HTML, CSS and JavaScript with hand-drawn SVG for trees and graphs, using a colour code that is consistent across views (token kinds, AST node categories, one colour per register, red for spills, green for auction winners, and a blue-to-red heat scale for block frequencies), so that the same colours used in the figures of this paper appear on screen. The GUI is started with a single start-up script, which launches the server on port 8765 and opens the browser, and every experiment in this section is run with one command-line invocation that writes JSON results.

A `unittest` suite of 27 tests covers the lexer, grammar, parser, semantic checks, SSA invariants (single assignment and dominance of uses), optimiser semantic preservation,



(b) Tokens view: counts by kind and the token stream of the maximal-munch DFA, coloured by kind

Simulated PG-RISC execution, k = 4 — profile: training input · measured: test input

Allocator	Cycles	Instrs	Loads	Stores	Spill mem-ops	Save/restore	Reg. util.	Code size	Spilled values	Realised gain	Output
PGARA (profiled)	**338,389**	**146,453**	**20,604**	**20,520**	**28,835**	**4,096**	**78.7%**	**146**	**24**	**321,749**	**✓ matches**
Static-Prior PGARA	358,163	142,292	28,603	20,458	36,772	4,096	77.0%	138	21	330,552	✓ matches
Loop-depth priority	414,979	166,692	36,707	28,562	52,980	4,096	74.6%	143	19	330,552	✓ matches
Linear scan	516,434	188,870	63,428	27,676	82,911	0	55.7%	166	29	162,534	✓ matches

(c) Run & Metrics view: the four allocators of the GUI compared on the test input

FIGURE 9. Two further views of the PGARA Compiler Studio on the rare-branch benchmark, cropped so that the text is legible in print. The AST and CFG views correspond to Figs. 2 and 4; full screenshots of all ten views are in the repository.

chordality, agreement of policy and value iteration, a hand-computed gain, the bound $V_{\text{reg}} \le V^*$, auction capacity, the Nash property, the exclusion of conceded coalitions, feasibility of every baseline, and end-to-end runs of all allocators on both benchmark suites; all tests pass. Measured with Coverage.py [55], the suite executes 94% of the statements of the compiler (97% of the allocator module), excluding only the experiment scripts. The interface was exercised end to end in headless Chromium through Playwright [56], which also produced the screenshots in Figs. 7 and 9.

Each program is compiled at the full optimisation level, profiled on a training input and measured on a different test input, for every allocator and $k \in \{3, 4, 5, 6, 8, 16, 32\}$. With 17 programs and seven allocators this gives 833 configurations, and the multiple-input study adds 595 more. Every simulated run is compared with the reference interpreter's output and all 1,428 match. Experiments ran on a Linux container with Python 3.13; because the simulator counts cycles deterministically, the results do not depend on the host.

B. PARAMETERS AND BENCHMARKS

Table 1 lists the parameters. The cost model matches the simulator's latencies, so the gains are expressed in the same cycles that are measured. Register budgets now extend to $k = 16$ and $k = 32$, the sizes of realistic RISC register files, to show what happens when pressure is low.

Programs come from three disjoint sets. The original suite of ten kernels (Table 2, top) spans straight-line code, regular loops, recursion and programs whose control flow depends on the data: a rare error branch, an early-exit search, a switch-like dispatch chain and a token scanner. The independent suite of seven programs (Table 2, bottom) was translated into MiniC from well-known embedded benchmarks after the original suite was fixed: CRC-32, bit counting, Dijkstra's shortest paths and quicksort after MiBench [57], Levenshtein distance and an FIR filter after BEEBS [58], and a 0/1 knapsack. MiniC has no bitwise operators, so shifts and XOR are written arithmetically. A separate tuning suite of five programs (bubble sort, prime counting, a histogram with outliers, naive string search and a gcd table) is used only to choose the hot-edge threshold ϑ and appears in no other result. On the tuning suite, at $k \in \{3, 4, 6\}$, the geometric mean of PGARA's cycles relative to linear scan is 0.803 for $\vartheta \in \{0.05, 0.1, 0.25\}$, 0.782 for $\vartheta = 0.5$, 0.781 for $\vartheta = 0.75$ and 0.850 without coalitions, so $\vartheta = 0.75$ is used throughout. (The first draft used $\vartheta = 0.25$, chosen on the evaluation suite itself.) Most programs generate their data with a linear congruential generator seeded from the input, so training and test inputs exercise the same code with different data; each program has five test inputs, and the first is used unless stated

TABLE 1. Parameter setup

Parameter	Value
Register budgets k	3, 4, 5, 6, 8, 16, 32
Load / store cost c_{ld} / c_{st} (cycles)	4 / 2
Register-access bonus σ, forwarded move c_{mv}	1 / 1
ALU / MUL / DIV latency	1 / 3 / 12
Taken / not-taken branch, call / return	2 / 1, 3 / 3
Discount γ	1 (fallback 0.999)
Static loop-stay probability	0.9
Hot-edge threshold ϑ (tuning suite)	0.75
Gain policy	always-in-register (14)
Optimiser rounds (max)	10
Interpreter / simulator step limits	$2 \cdot 10^7$ / $8 \cdot 10^7$
Programs: original / independent / tuning	10 / 7 / 5
Test inputs per program	5 (training $\neq$ test)

TABLE 2. Benchmarks: SSA values, φs, largest interference clique ω, mean and maximum policy-iteration rounds, profiling time (interpreter on the training input) and PGARA allocation time (k = 4). Top: original suite; bottom: independent suite

Benchmark	$\lvert V \rvert$	φ	ω	PI_{mean}	PI_{max}	prof. ms	alloc. ms
straightline	39	0	11	1.00	1	0	1.2
arraysum	38	16	9	1.29	3	67	2.9
matmul	51	10	8	1.27	3	116	2.9
recursion	39	4	7	1.28	3	22	2.3
rarebranch	62	30	12	1.26	3	73	9.2
earlyexit	41	17	10	1.29	3	112	4.1
dispatch	116	55	15	1.17	7	45	16.2
scanner	89	49	11	1.15	4	88	11.3
sort	53	16	10	1.32	3	211	4.4
collatz	30	16	9	1.53	3	181	5.6
crc	50	20	13	1.38	6	506	7.2
bitcount	52	18	10	1.44	11	114	8.6
dijkstra	61	19	11	1.41	6	469	10.5
levenshtein	65	16	14	1.29	6	723	18.9
knapsack	48	8	8	1.15	2	342	2.9
fir	60	21	11	1.23	3	354	5.3
qsort	72	15	8	1.57	10	48	14.6

otherwise.

C. BASELINES AND METRICS

We compare seven allocators that share the front end, optimiser, interference graph and code generator, so that only the allocation decision differs: (i) PGARA (PG) with profiled probabilities; (ii) Static-Prior PGARA (SP), the same engine with (9); (iii) Profiled-frequency priority (FP), an ablation that keeps PGARA's coalitions, auction, binding and best response but replaces the MDP gain by the access count weighted by the profiled block frequency ϕ of (13), i.e. the 10^{depth} weight of (iv) replaced by measured frequency; (iv) Loop-depth priority (LD), in the style of Chow and Hennessy [10], which weights each access by 10^{depth} and runs the same auction and binding; (v) Chaitin–Briggs with profiled spill costs (CBp) [9], [43]: conservative (Briggs) coalescing of φ-related values, simplification with optimistic spilling by lowest cost/degree, and biased colouring towards φ partners, with spill costs (loads, stores and forwarded moves) weighted by the profiled block frequency, which is how a production allocator uses a profile [11]; (vi) Chaitin–Briggs with static costs (CB), the same with 10^{depth} weights; and (vii) Linear scan (LS) [13] over live intervals in layout order, with register hints for φ partners. FP and CBp answer the question whether PGARA's gains are worth more than simply weighting accesses by measured frequency; SP answers whether the profile is worth anything. PG-RISC is not an LLVM target, so LLVM's own allocators are represented by these re-implementations rather than run directly; this keeps every other factor identical.

We report five metrics: (1) simulated cycles on the test input; (2) dynamic spill memory operations, the loads and stores of spill slots executed; (3) register utilisation, $1 -$ spill memops/value accesses, the fraction of operand accesses served from registers; (4) static code size in instructions; and (5) compile cost, separated into the profiling run and the allocation itself (Table 2 and Section IV-F).

D. EXECUTION TIME

Table 3 lists cycles at $k = 4$ for all programs and allocators, and Fig. 10 shows the four profile-aware or profile-ablated allocators normalised to linear scan. PGARA is fastest or tied on 13 of the 17 programs, and its geometric mean is 21.8% below linear scan, 2.6% below Static-Prior, 3.3% below profiled-frequency priority, 3.7% below Chaitin–Briggs with profiled costs and 5.2% below loop-depth priority. The largest gains are on programs with skewed, data-dependent branches. On rare-branch, PGARA needs 5.5% fewer cycles than Static-Prior, 11.7% fewer than profiled-frequency priority and 3.8% fewer than profiled Chaitin–Briggs, because only the MDP charges the error-path counters for what they actually cost: the frequency weight alone still lets the error path's several accesses add up to a competitive bid. On the independent suite the pattern repeats: Dijkstra (13% below FP and 15% below CBp), knapsack (9%) and the FIR filter (2–8%) favour PGARA. PGARA is not uniformly best. On quicksort, PGARA is 4.6% slower than profiled-frequency priority and 3.6% slower than Static-Prior: the recursive partition loop has a phase behaviour that average-case probabilities blur. On Levenshtein, loop-depth priority is 0.3% faster, and Static-Prior is 31% slower because the static prior sees the match/mismatch branch as a fair coin. Straight-line code favours linear scan by 1.4%: with a single block there is no uncertainty to exploit.

Across register budgets (Table 4 and Fig. 11) PGARA has the lowest geometric-mean cycle count at every $k \leq 8$: 19–22% below linear scan, 1.1–3.1% below Static-Prior, 3.3–4.8% below profiled-frequency priority, 1.2–4.5% below profiled Chaitin–Briggs and 4.8–6.4% below loop-depth priority. The advantage holds on both suites separately: at $k = 4$ it is 2.9% over FP and 2.9% over CBp on the original suite, and 3.8% and 4.9% on the independent suite. It is not uniform, however. At $k = 3$, profiled Chaitin–Briggs is 20.7% faster on matmul and about 12% faster on Levenshtein and the FIR filter: under extreme pressure the auction decides each clique greedily in MCS order and keeps coalitions whole, whereas optimistic colouring with coalescing finds a tighter packing.

TABLE 3. Cycles ($\times 10^3$) on the primary test input at k = 4. Bold: best. PG: PGARA; SP: Static-Prior PGARA; FP: profiled-frequency priority; LD: loop-depth priority; CBp / CB: Chaitin–Briggs with profiled / static spill costs; LS: linear scan. Top: original suite; bottom: independent suite

Benchmark	PG	SP	FP	LD	CBp	CB	LS
straightline	0.286	0.286	0.286	0.286	0.286	0.286	**0.282**
arraysum	**311.4**	**311.4**	**311.4**	**311.4**	315.4	315.4	344.8
matmul	**565.5**	**565.5**	**565.5**	**565.5**	583.7	583.7	817.8
recursion	**201.6**	**201.6**	202.1	202.1	202.2	202.2	201.7
rarebranch	**338.4**	358.2	383.2	415.0	351.7	424.8	516.4
earlyexit	650.6	**650.4**	651.5	652.1	652.0	652.0	676.2
dispatch	**138.8**	**138.8**	153.2	171.7	140.1	156.4	236.4
scanner	**411.1**	479.0	431.0	480.8	474.9	516.5	530.7
sort	**849.5**	**849.5**	868.8	868.8	894.4	894.4	988.4
collatz	**655.3**	**655.3**	657.2	657.2	657.2	657.2	942.2
crc	**4009.3**	**4009.3**	4026.5	4026.5	4036.5	4036.5	4826.2
bitcount	**629.7**	**629.7**	633.2	632.9	632.1	632.1	716.8
dijkstra	**1242.1**	**1242.1**	1429.3	1474.0	1456.3	1474.0	1661.9
levenshtein	2721.6	3567.6	2862.3	**2714.2**	2717.9	2717.9	3673.8
knapsack	**734.7**	**734.7**	805.1	805.1	805.2	805.2	966.5
fir	**1155.2**	1167.5	1179.7	1179.8	1249.5	1249.5	1895.6
qsort	219.9	212.3	**210.2**	223.4	222.8	219.2	293.8
geo-mean / LS	0.782	0.803	0.808	0.825	0.812	0.830	1.000

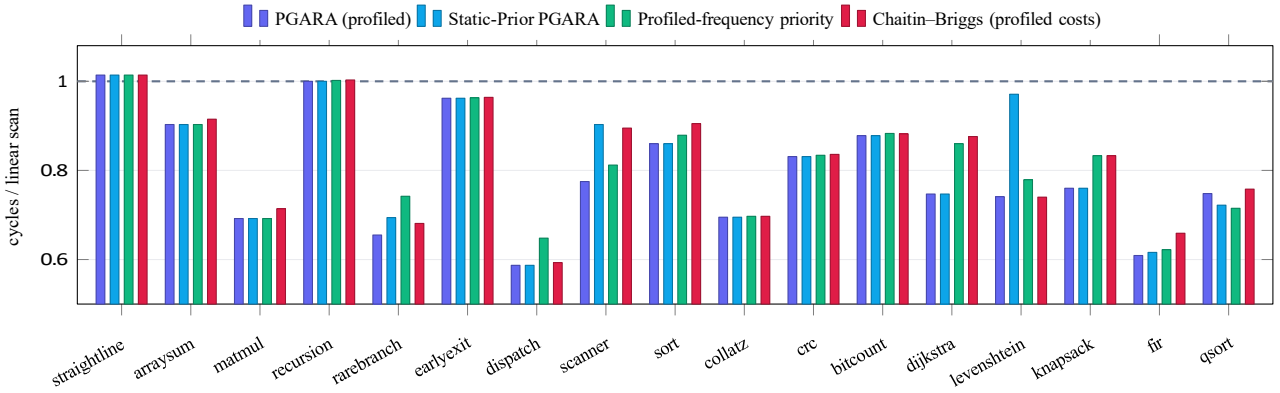


FIGURE 10. Simulated cycles at k = 4 on the test inputs, normalised to linear scan (dashed grey line, 1.0); lower is better. The first ten programs are the original suite, the last seven the independent suite. The PGARA colours match the GUI's Run & Metrics view.

These are the cases where PGARA is beaten by more than 5%.

At $k = 16$ and $k = 32$ register pressure disappears: no auction-based allocator spills at all, so PGARA, Static-Prior and both priority allocators produce identical code. Chaitin–Briggs is 0.3% faster there, because its coalescing removes a few φ copies that PGARA's binding hints leave, while PGARA still needs 16% and 11% fewer cycles than linear scan, which spills at interval boundaries. The benefit of PGARA therefore lies in the range where registers are scarce, which on real RISC targets corresponds to code with high register pressure (large loop bodies, unrolled or inlined code) rather than to typical small functions. Results are not strictly monotonic in k for every program, because each extra register that holds a value across a call adds caller-save traffic.

E. SPILL TRAFFIC AND REGISTER UTILISATION

Dynamic spill traffic (Fig. 12) is where the expected-gain objective shows most directly, because (14) is precisely the expected spill traffic avoided. Summed over the 17 programs at $k = 4$, PGARA executes 50% fewer spill memory operations than linear scan, 23% fewer than Static-Prior, 12% fewer than profiled-frequency priority and 15% fewer than profiled Chaitin–Briggs; at $k = 8$ it nearly eliminates spill traffic (68k operations against 137k, 205k, 172k and 593k). The exception is again $k = 3$, where profiled Chaitin–Briggs executes 1.5% fewer spill operations than PGARA. Register

TABLE 4. Geometric-mean cycles over all 17 programs, relative to linear scan, by register budget. Bold: best

k	PG	SP	FP	LD	CBp	CB	LS
3	**0.795**	0.821	0.824	0.846	0.804	0.824	1.000
4	**0.782**	0.803	0.808	0.825	0.812	0.830	1.000
5	**0.804**	0.813	0.836	0.852	0.842	0.853	1.000
6	**0.805**	0.820	0.846	0.861	0.842	0.852	1.000
8	**0.807**	0.819	0.842	0.848	0.837	0.841	1.000
16	0.843	0.843	0.843	0.843	**0.841**	**0.841**	1.000
32	0.891	0.891	0.891	0.891	**0.888**	**0.888**	1.000

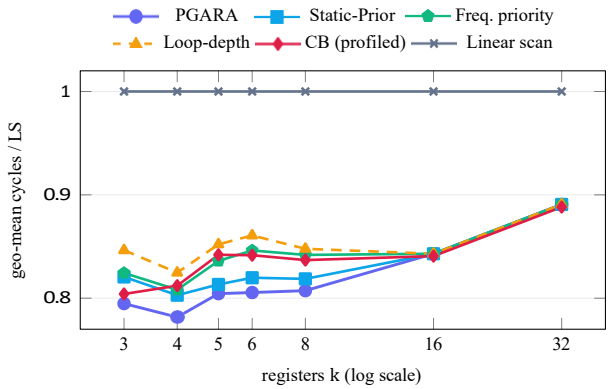


FIGURE 11. Geometric-mean cycles over the 17 programs, normalised to linear scan, as the register budget varies from 3 to 32.

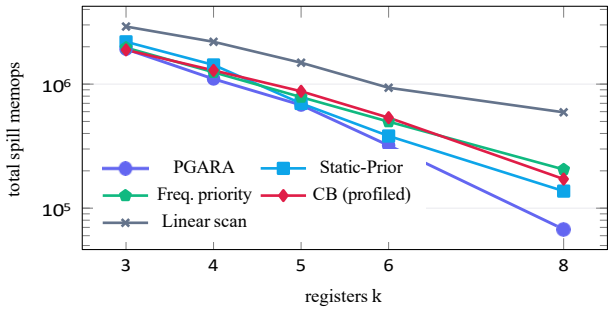


FIGURE 12. Dynamic spill loads and stores summed over the 17 programs (log scale). At $k \geq 16$ only linear scan spills.

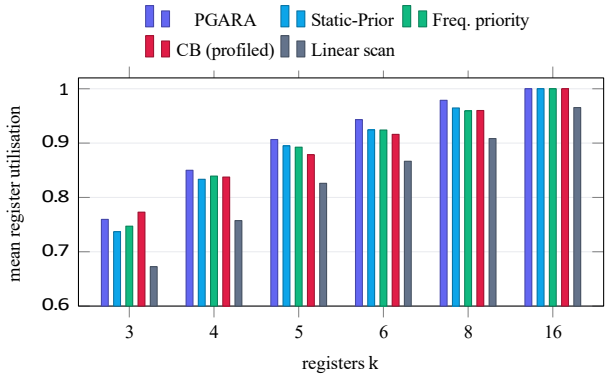


FIGURE 13. Mean register utilisation (fraction of operand accesses served from registers) over the 17 programs.

TABLE 5. Static code size (instructions) summed over the 17 programs

k	PG	SP	FP	LD	CBp	CB	LS
3	**2660**	2662	2684	2706	2677	2668	2946
4	**2571**	2585	2606	2611	2574	2593	2831
5	2491	**2484**	2529	2511	2501	2514	2769
6	2418	**2402**	2472	2464	2453	2459	2713
8	2308	**2303**	2343	2333	2309	2309	2660
16	2180	2180	2180	2180	**2097**	**2097**	2623
32	2180	2180	2180	2180	**2097**	**2097**	2567

utilisation (Fig. 13) follows: the mean fraction of operand accesses served from registers rises from 76% at $k = 3$ to 98% at $k = 8$ for PGARA, against 67% to 91% for linear scan, and reaches 100% for every allocator except linear scan at $k = 16$. The number of static spilled values can be higher for PGARA than for the priority allocators: PGARA deliberately spills many cold values in order to keep the few hot ones, which is the intended behaviour of a gain-maximising allocator.

F. CODE SIZE AND COMPILE COST

Table 5 gives static code size summed over the 17 programs. The auction-based allocators and Chaitin–Briggs produce code of similar size, 9–17% smaller than linear scan, because shared web slots, coalescing and register hints remove most φ copies, and fewer spilled operands need explicit loads and stores. Code size is not the objective of PGARA; it is smallest at $k = 3, 4$, Static-Prior is marginally smaller at $k = 5$–8, and Chaitin–Briggs, whose coalescing is more aggressive, is 4% smaller at $k \geq 16$.

The compile cost of PGARA has two parts, which Table 2 reports separately. The profiling run, the reference interpreter executing the training input, takes 0.1–723 ms per program and dominates the total: summed over the 17 programs it is 27 times the allocation time. It runs once per program and training input, independently of k and of the allocator, and its cost is that of executing the program; in a production setting it is replaced by an instrumented or sampled run as in any profile-guided build. The allocation itself takes 1.2–18.9 ms per program in pure Python. Policy iteration needs 1.29 improvement rounds on average over the 966 values of both suites, with a maximum of 11, and every interference graph in the study was verified chordal, with the largest clique at 15 values.

To see how allocation time grows with function size, we generated two families of programs (Table 6). In the wide family a loop updates m accumulators split over two branches, so the number of values grows with m while live ranges stay short (at most six blocks). PGARA's time grows moderately, roughly as $|V|^{1.35}$, from 6.9 ms for 37 values to 125 ms for 906 values, and above about 450 values it is lower than that of Chaitin–Briggs, whose simplification loop is quadratic in our implementation. In the long family a loop body is a chain of m conditionals, so every accumulator is live across the whole body and both $|V|$ and $|L_v|$ grow with m. Here the cost of the per-value linear systems shows: time grows roughly as $(\sum_v |L_v|)^{1.4}$, reaching 2.4 s for 398 values whose live ranges span up to 291 blocks, against 41 ms for Chaitin–Briggs. Long live ranges are therefore PGARA's main scalability limit. The systems are banded in reverse postorder and their solutions change little between policy-iteration rounds, so a banded or incremental solver, or sharing one factorisation between values with the same live range, would remove most of this cost; we leave this to future work.

TABLE 6. Allocation time (ms) on generated programs at k = 8: values |V|, total and maximum live-range length in blocks

family	m	$\|V\|$	$\sum\|L_v\|$	$\max\|L_v\|$	PG	CBp	LS
wide	8	37	80	6	6.9	0.9	0.3
wide	16	66	141	6	4.0	1.4	0.2
wide	32	122	261	6	8.2	4.2	0.4
wide	64	234	501	6	17.5	14.4	0.7
wide	128	458	981	6	39.3	52.3	1.4
wide	256	906	1941	6	124.6	237.3	3.5
long	4	21	110	15	2.7	0.3	0.1
long	8	37	310	27	7.0	0.6	0.2
long	16	76	1194	51	31.2	2.0	0.6
long	32	142	4156	99	151.5	5.9	1.5
long	64	270	14684	195	891.9	20.2	5.2
long	96	398	31356	291	2411.8	41.2	10.5

TABLE 7. Geometric-mean cycles relative to linear scan at k = 4 over the five test inputs: mean, minimum and maximum across inputs

Allocator	mean	min	max
PG	0.7821	0.7811	0.7830
SP	0.8032	0.8022	0.8041
FP	0.8084	0.8072	0.8091
LD	0.8245	0.8241	0.8248
CBp	0.8121	0.8114	0.8127
CB	0.8304	0.8300	0.8309
LS	1.0000	1.0000	1.0000

G. ABLATION: WHAT THE MDP ADDS BEYOND PROFILING

Two comparisons isolate the contributions. Static-Prior uses the MDP, coalitions and auction without a profile, so PG versus SP measures the profile: 1.1–3.1% at $k \le 8$. Profiled-frequency priority uses the profile, coalitions, auction and best response but no MDP, so PG versus FP measures the MDP gain on top of profile-weighted access counts: 3.3–4.8% at $k \le 8$, with 12–14% fewer spill operations at $k = 4$–5. Profiled Chaitin–Briggs uses the profile in the way production compilers do, through frequency-weighted spill costs, with a different allocation algorithm; PG versus CBp is 1.2–4.5% at $k \le 8$. Since FP and CBp see exactly the same profile as PGARA, these differences are due to what the MDP models and frequency weighting does not: the reload after a spilled definition that the peephole pass forwards, the save and restore around calls, and the copy through memory when a loop-carried value is split. Where these effects are absent, the allocators agree: on straight-line code, arraysum, matmul and collatz, PG and FP produce the same code at $k = 4$.

H. ROBUSTNESS

Several test inputs. Table 7 repeats the $k = 4$ study with five test inputs per program (85 runs per allocator). The geometric means move by at most 0.25% between inputs and the ranking of the allocators never changes; per program, PGARA's ratio to linear scan varies by at most 2.2 percentage points (dispatch, 0.575–0.598), and PGARA is best or tied in 64 of the 85 runs.

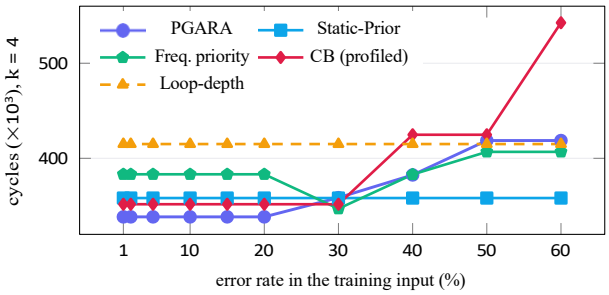


FIGURE 14. Rare-branch at k = 4, measured on a test input with a 1% error rate, as the error rate of the training input grows. Linear scan (516k cycles) is omitted for clarity.

TABLE 8. PGARA's geometric-mean cycles relative to linear scan at k = 4 for different model constants

σ	$c_{mv} = 0$	$c_{mv} = 1$	$c_{mv} = 2$
0	0.7801	0.7803	0.7809
0.5	0.7803	0.7817	0.7830
1	0.7803	0.7817	0.7830
2	0.7817	0.7830	0.7830

Quality of the training profile. A profile-driven allocator is only as good as its profile. Fig. 14 varies the error rate of the rare-branch training input from 1% to 60% while the test input keeps its 1% error rate. At $k = 4$, PGARA's allocation is unchanged and best of all allocators up to a training error rate of 20%; at 30% it ties Static-Prior (profiled-frequency priority is 3% faster at that point); from 40% the profile misleads it and it falls behind Static-Prior (by 6.8% at 40% and 16.9% at 50–60%) and, at 50%, 0.9% behind loop-depth priority, while staying ahead of linear scan. At $k = 8$ the allocation is unchanged up to 40%. The degradation is therefore a step function rather than gradual, because an allocation changes only when the ranking of bids changes, and the profile can be wrong by a large factor before it does. The training input must still resemble production behaviour, as with any profile-guided optimisation [23], [26], and Static-Prior is the right fallback when no such input exists.

Model constants. Varying the register-access bonus σ over $\{0, 0.5, 1, 2\}$ and the forwarded-move cost c_{mv} over $\{0, 1, 2\}$ changes PGARA's geometric mean at $k = 4$ only between 0.780 and 0.783 (Table 8) and leaves it unchanged at $k = 8$, so the conclusions do not depend on these constants.

Realised versus optimal gain. Over the 966 values of both suites, the optimal policy V^* would spill or reload inside the live range for 189 values (20%), but the realised gain (14) differs from the optimal-policy gain by only 0.05% in total, and by more than 1% for 9 of the 809 values whose gain exceeds one cycle. Bidding with either gain gives identical cycle counts in 48 of 51 configurations ($k \in \{3, 4, 8\}$) and at most 0.5% difference in the others. The bid of the first draft was therefore nearly harmless on these programs, but the realised gain is the correct quantity and is used throughout this version.

TABLE 9. Qualitative comparison of allocation approaches

Approach	Decision basis	Measured profile	Training	Explainable
Linear scan [13]	interval order	no	none	partly
Loop-depth priority [10]	10^d weights	no	none	yes
Chaitin–Briggs [43]	frequency-weighted spill cost	yes, as weights	none	partly
LLVM greedy [11]	block-frequency spill weight, eviction	yes, as weights	none	partly
RL4ReAl [19]	learned policy	indirectly	heavy	no
PGARA (this work)	expected gain $G(v)$, auction	yes, in the MDP	none	yes (bids)

Coalition repair. Concessions happen in practice: across the 119 configurations of the main study, 105 values conceded their registers in the repair step, and the exclusion set of Algorithm 3 kept every one of them out of the second best-response pass.

I. COMPARISON WITH RELATED APPROACHES

Table 9 positions PGARA against the allocators it was measured against and against the learned allocator RL4ReAl [19], which we compare qualitatively because it targets LLVM's back ends.

J. ANSWERS TO THE RESEARCH QUESTIONS

RQ1: Does profiling help? Yes, moderately and where control flow depends on data. With a representative training input, PGARA is 1.1–3.1% faster than the same engine with static priors at $k \le 8$ (geometric mean over 17 programs) and executes 3–51% fewer spill operations; the gain reaches 14% on scanner and 24% on Levenshtein. With a training input whose branch behaviour differs strongly from the test input (above about 30% error rate where the test has 1%), static priors are better.

RQ2: Does the MDP and auction formulation help beyond profiling? Yes. With the same profile, PGARA is 3.3–4.8% faster than profiled-frequency priority, which differs only in replacing the MDP gain by frequency-weighted access counts, and 1.2–4.5% faster than Chaitin–Briggs with profiled spill costs, at every $k \le 8$ and on both suites. The exception is extreme pressure ($k = 3$) on three programs, where Chaitin–Briggs packs registers better.

RQ3: How robust are the results? The ranking is stable across five test inputs per program, across the original and the independent suites, and across the model constants. The benefit vanishes at $k \ge 16$, where no allocator except linear scan spills on these programs, and it depends on a representative profile, with a step-shaped degradation beyond a 20–40% error rate.

RQ4: What does it cost? Allocation takes 1–19 ms per program and grows roughly as $|V|^{1.35}$ with the number of values when live ranges are short, but super-linearly, about $(\sum_v |L_v|)^{1.4}$, when live ranges span hundreds of blocks. The profiling run costs more than the allocation (27 times as much in total here) but is shared by all register budgets and is the standard cost of any profile-guided build.

K. THREATS TO VALIDITY

Benchmarks. The programs are small kernels: ten written for this study and seven translated from MiBench and BEEBS. The translation replaces bitwise operations with arithmetic, which changes their instruction mix, and none of the programs is a large application. ϑ is now tuned on a separate suite, but the cost constants of Table 1 are shared with the simulator. *Simulator.* Cycles come from a simulator with a fixed latency table rather than from hardware; PG-RISC has no caches and no branch predictor, so a spill always costs the same and a mispredicted branch costs nothing extra, both of which favour a model whose costs are exact. *Register budgets.* The study covers $k \le 32$, but the benefit is confined to $k \le 8$ on these programs; whether it appears at realistic budgets depends on how often real code reaches that pressure. *Baselines.* The Chaitin–Briggs and profiled-frequency baselines are our re-implementations on PG-RISC; they include conservative coalescing, optimistic spilling and biased colouring but not live-range splitting or the eviction and region splitting of LLVM's greedy allocator, so a production allocator could be stronger. *Splitting.* The MDP's optimal policy may move a value between register and memory inside its live range; the code generator does not split live ranges, so this part of the policy is computed but not realised, and the bid now uses the realised policy instead.

V. CONCLUSION AND FUTURE WORK

PGARA allocates registers adaptively, guided by measured profiles, by expected gain: a per-value Markov decision process over the live range, evaluated with measured branch probabilities under the policy the code generator realises, followed by a sealed-bid auction among φ-web coalitions on the chordal SSA interference graph and best-response dynamics to a Nash equilibrium. Existing allocators rank values by static weights that cannot see data-dependent branches, scale access counts by profiled frequencies without modelling where spills and reloads occur, or learn opaque policies from large corpora.

The method makes the expected saving of each decision an explicit number, needs no training, and costs milliseconds for ordinary functions. It is embedded in a complete compiler, from an LL(1) grammar and attribute-grammar semantic analysis through TAC, SSA and optimisation to PG-RISC code and a simulator, with a web interface that shows every phase.

On 17 programs and seven register budgets, all compiled programs match the reference output. At $k = 4$, PGARA reduces geometric-mean cycles by 21.8% relative to linear scan, 3.7% relative to Chaitin–Briggs with profile-weighted spill costs, 3.3% relative to profiled-frequency priority and 2.6% relative to static priors, and it executes half as many spill memory operations as linear scan. The ablations show

that profiling and the MDP gain each contribute, that the benefit is concentrated on programs with skewed, data-dependent control flow and on budgets of at most eight registers, and that it disappears when the training profile misrepresents the test behaviour by a large margin. Under extreme pressure ($k = 3$), optimistic colouring with coalescing still packs some programs better, and long live ranges make the per-value linear systems the dominant compile-time cost.

Future work follows from these limitations: extend the MDP's policy into live-range splitting in the code generator so that values can move between register and memory inside a range; replace dense elimination by banded or incremental solvers for long live ranges; combine the gain with optimistic colouring when pressure is extreme; replace whole-program averages with context-sensitive or phase-aware profiles where the profile misleads; port the engine to a production back end such as LLVM to evaluate it on standard suites and real hardware; and combine measured profiles with learned profile prediction [25] when no representative input is available.

AUTHOR CONTRIBUTIONS

1) **Shreyans Chowdry**: core pipeline design, lexical and syntax analysis, AST construction and intermediate code generation, and the web GUI and pipeline visualisation interface, including the Allocation view that displays gains, bids, coalitions and auction rounds; in the PGARA allocator, the edge profiler interface, the transition matrices (8)–(9), the block-frequency solver (13) and the hot-edge φ-web coalitions and bids (15)–(16).
2) **Samrender Singh Saini**: semantic analysis, scope resolution and symbol table management; in the PGARA allocator, the interference graph, the maximum cardinality search and PEO verification, and the sealed-bid auction and register binding (Algorithm 2).
3) **Karthikeya Jami**: SSA construction, the optimisation engine and PG-RISC code generation; in the PGARA allocator, the residency MDP, policy iteration and the gain computation (Algorithm 1, (10)–(14)), and the integration of the allocation with code generation.
4) **Abhishek Kumar**: runtime environment, simulator, test suite and benchmarking; in the PGARA allocator, best-response dynamics, coalition repair (Algorithm 3), the baseline allocators and ablations used in Section IV, and the experiment scripts.
5) **Nishanth Kumar**: assisted with the user documentation and with testing the web interface.

All authors took part in the design discussions of the PGARA allocator and in the writing of this manuscript.

ACKNOWLEDGEMENTS

The authors thank Bhuvaneswari M, School of Computer Science and Engineering, Vellore Institute of Technology, for guidance on recasting the allocator around stochastic dynamic programming and game theory, and the reviewer of the first draft for comments that led to the additional baselines and experiments in this version.

FUNDING

This work received no external funding.

COMPETING INTERESTS

The authors declare no competing interests.

DATA AND CODE AVAILABILITY

The compiler, the web interface, the three benchmark suites, the unit tests, the experiment scripts and the raw results (JSON for all 1,428 configurations and every study in Section IV) are publicly available at https://github.com/shreyans-chowdry/pgara. The results in this paper were produced from commit `9b154888a712f99e2170fea9a797388995d731f5` (branch `draft2-revision`) with `python3 -m pgara.experiments`; the full-window screenshots of every view of the Compiler Studio are in `docs/screenshots`, and a live instance of the interface is hosted at https://pgara.vercel.app.

REFERENCES

[1] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO), 2021, pp. 2–14.
[2] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," Proc. IEEE, vol. 106, no. 11, pp. 1879–1901, 2018.
[3] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," ACM Comput. Surv., vol. 51, no. 5, art. 96, 2018.
[4] H. Leather and C. Cummins, "Machine learning in compilers: Past, present and future," in Proc. Forum for Specification and Design Languages (FDL), 2020.
[5] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," ACM Trans. Program. Lang. Syst., vol. 13, no. 4, pp. 451–490, 1991.
[6] F. Rastello and F. Bouchez Tichadou, Eds., SSA-based Compiler Design. Cham, Switzerland: Springer, 2022.
[7] K. D. Cooper and L. Torczon, Engineering a Compiler, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.
[8] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.
[9] G. J. Chaitin, "Register allocation & spilling via graph coloring," in Proc. SIGPLAN Symp. Compiler Construction, 1982, pp. 98–105.
[10] F. C. Chow and J. L. Hennessy, "The priority-based coloring approach to register allocation," ACM Trans. Program. Lang. Syst., vol. 12, no. 4, pp. 501–536, 1990.
[11] J. Olesen, "Greedy register allocation in LLVM 3.0," LLVM Project Blog, Sep. 2011. [Online]. Available: https://blog.llvm.org/2011/09/greedy-register-allocation-in-llvm-30.html
[12] LLVM Project, "LLVM block frequency terminology," LLVM documentation. [Online]. Available: https://llvm.org/docs/BlockFrequencyTerminology.html
[13] M. Poletto and V. Sarkar, "Linear scan register allocation," ACM Trans. Program. Lang. Syst., vol. 21, no. 5, pp. 895–913, 1999.
[14] S. Hack, D. Grund, and G. Goos, "Register allocation for programs in SSA-form," in Proc. Int. Conf. Compiler Construction (CC), LNCS 3923, 2006, pp. 247–262.
[15] S. Hack and G. Goos, "Optimal register allocation for SSA-form programs in polynomial time," Inf. Process. Lett., vol. 98, no. 4, pp. 150–155, 2006.

[16] R. Castañeda Lozano, M. Carlsson, G. Hjort Blindell, and C. Schulte, "Combinatorial register allocation and instruction scheduling," ACM Trans. Program. Lang. Syst., vol. 41, no. 3, art. 17, 2019.
[17] R. Castañeda Lozano and C. Schulte, "Survey on combinatorial register allocation and instruction scheduling," ACM Comput. Surv., vol. 52, no. 3, art. 62, 2019.
[18] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: A machine learning guided compiler optimizations framework," arXiv:2101.04808, 2021.
[19] S. VenkataKeerthy, S. Jain, A. Kundu, R. Aggarwal, A. Cohen, and R. Upadrasta, "RL4ReAl: Reinforcement learning for register allocation," in Proc. ACM SIGPLAN Int. Conf. Compiler Construction (CC), 2023, pp. 133–144.
[20] S. VenkataKeerthy, R. Aggarwal, S. Jain, M. S. Desarkar, R. Upadrasta, and Y. N. Srikant, "IR2Vec: LLVM IR based scalable program embeddings," ACM Trans. Archit. Code Optim., vol. 17, no. 4, art. 32, 2020.
[21] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "CompilerGym: Robust, performant compiler optimization environments for AI research," in Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO), 2022, pp. 92–105.
[22] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. P. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in Proc. Int. Conf. Machine Learning (ICML), PMLR 139, 2021, pp. 2244–2253.
[23] M. Panchenko, R. Auler, B. Nell, and G. Ottoni, "BOLT: A practical binary optimizer for data centers and beyond," in Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO), 2019, pp. 2–14.
[24] H. Shen, K. Pszeniczny, R. Lavaee, S. Kumar, S. Tallam, and X. D. Li, "Propeller: A profile guided, relinking optimizer for warehouse-scale applications," in Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2023.
[25] N. Rotem and C. Cummins, "Profile guided optimization without profiles: A machine learning approach," arXiv:2112.14679, 2021.
[26] T. A. Wagner, V. Maverick, S. L. Graham, and M. A. Harrison, "Accurate static estimators for program optimization," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 1994, pp. 85–96.
[27] Y. Wu and J. R. Larus, "Static branch frequency and program profile analysis," in Proc. 27th Annu. Int. Symp. Microarchitecture (MICRO), 1994, pp. 1–11.
[28] D. W. Wall, "Predicting program behavior using real or estimated profiles," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 1991, pp. 59–70.
[29] D. Callahan and B. Koblenz, "Register allocation via hierarchical graph coloring," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 1991, pp. 192–203.
[30] D. R. Koes and S. C. Goldstein, "A global progressive register allocator," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 2006, pp. 204–215.
[31] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
[32] M. L. Puterman, Markov Decision Processes: Discrete Stochastic Dynamic Programming. New York, NY, USA: Wiley, 1994.
[33] D. P. Bertsekas, Reinforcement Learning and Optimal Control. Belmont, MA, USA: Athena Scientific, 2019.
[34] R. A. Howard, Dynamic Programming and Markov Processes. Cambridge, MA, USA: MIT Press, 1960.
[35] L. S. Shapley, "A value for *n*-person games," in Contributions to the Theory of Games II, Annals of Mathematics Studies 28. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.
[36] W. Vickrey, "Counterspeculation, auctions, and competitive sealed tenders," J. Finance, vol. 16, no. 1, pp. 8–37, 1961.
[37] R. E. Tarjan and M. Yannakakis, "Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs," SIAM J. Comput., vol. 13, no. 3, pp. 566–579, 1984.
[38] R. W. Rosenthal, "A class of games possessing pure-strategy Nash equilibria," Int. J. Game Theory, vol. 2, no. 1, pp. 65–67, 1973.
[39] N. Nisan, T. Roughgarden, É. Tardos, and V. V. Vazirani, Eds., Algorithmic Game Theory. Cambridge, U.K.: Cambridge Univ. Press, 2007.
[40] S. Jain, Y. Andaluri, S. VenkataKeerthy, and R. Upadrasta, "POSET-RL: Phase ordering for optimizing size and execution time using reinforcement learning," in Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS), 2022, pp. 121–131.
[41] A. Haj-Ali, N. K. Ahmed, T. Willke, Y. S. Shao, K. Asanovic, and I. Stoica, "NeuroVectorizer: End-to-end vectorization with deep reinforcement learning," in Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO), 2020, pp. 242–255.
[42] C. Cummins, V. Seeker, D. Grubisic, B. Rozière, J. Gehring, G. Synnaeve, and H. Leather, "LLM Compiler: Foundation language models for compiler optimization," in Proc. 34th ACM SIGPLAN Int. Conf. Compiler Construction (CC), 2025.
[43] P. Briggs, K. D. Cooper, and L. Torczon, "Improvements to graph coloring register allocation," ACM Trans. Program. Lang. Syst., vol. 16, no. 3, pp. 428–455, 1994.
[44] L. George and A. W. Appel, "Iterated register coalescing," ACM Trans. Program. Lang. Syst., vol. 18, no. 3, pp. 300–324, 1996.
[45] G.-Y. Lueh, T. Gross, and A.-R. Adl-Tabatabai, "Fusion-based register allocation," ACM Trans. Program. Lang. Syst., vol. 22, no. 3, pp. 431–470, 2000.
[46] D. E. Knuth, "On the translation of languages from left to right," Inf. Control, vol. 8, no. 6, pp. 607–639, 1965.
[47] P. M. Lewis and R. E. Stearns, "Syntax-directed transduction," J. ACM, vol. 15, no. 3, pp. 465–488, 1968.
[48] D. E. Knuth, "Semantics of context-free languages," Math. Syst. Theory, vol. 2, no. 2, pp. 127–145, 1968.
[49] K. D. Cooper, T. J. Harvey, and K. Kennedy, "A simple, fast dominance algorithm," Rice Univ., Houston, TX, USA, Tech. Rep., 2001.
[50] M. N. Wegman and F. K. Zadeck, "Constant propagation with conditional branches," ACM Trans. Program. Lang. Syst., vol. 13, no. 2, pp. 181–210, 1991.
[51] R. Bellman, Dynamic Programming. Princeton, NJ, USA: Princeton Univ. Press, 1957.
[52] P. N. Panagopoulou and P. G. Spirakis, "A game theoretic approach for efficient graph coloring," in Proc. Int. Symp. Algorithms and Computation (ISAAC), LNCS 5369, 2008, pp. 183–195.
[53] D. Monderer and L. S. Shapley, "Potential games," Games Econ. Behav., vol. 14, no. 1, pp. 124–143, 1996.
[54] Python Software Foundation, "Python 3 documentation," 2024. [Online]. Available: https://docs.python.org/3/
[55] N. Batchelder et al., "Coverage.py: Code coverage measurement for Python." [Online]. Available: https://coverage.readthedocs.io/
[56] Microsoft, "Playwright: Fast and reliable end-to-end testing for modern web apps," 2024. [Online]. Available: https://playwright.dev/
[57] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in Proc. IEEE Int. Workshop Workload Characterization (WWC-4), 2001, pp. 3–14.
[58] J. Pallister, S. Hollis, and J. Bennett, "BEEBS: Open benchmarks for energy measurements on embedded platforms," arXiv:1308.5174, 2013.



SHREYANS CHOWDRY is pursuing the B.Tech. degree in Computer Science and Engineering with a specialisation in Data Science at the Vellore Institute of Technology, Vellore, India. His interests include compilers, applied artificial intelligence and full-stack software engineering. In this work he designed the core compiler pipeline (lexer, LL(1) grammar and parser, AST and three-address code), built the web interface, and developed the profile-derived transition model and the φ-web coalitions of the PGARA allocator.



SAMRENDER SINGH SAINI is pursuing the B.Tech. degree in Computer Science and Engineering at the Vellore Institute of Technology, Vellore, India. In this work he developed the semantic analysis, scope resolution and symbol table management of the compiler, and the interference graph, maximum cardinality search and sealed-bid auction of the PGARA allocator.



KARTHIKEYA JAMI is pursuing the B.Tech. degree in Computer Science and Engineering at the Vellore Institute of Technology, Vellore, India. In this work he developed SSA construction, the optimiser and PG-RISC code generation, and the residency MDP, policy iteration and gain computation of the PGARA allocator.



ABHISHEK KUMAR is pursuing the B.Tech. degree in Computer Science and Engineering at the Vellore Institute of Technology, Vellore, India. In this work he developed the reference interpreter, the cycle-counting simulator, the test suite and the benchmark harness, and the best-response dynamics, coalition repair and baseline allocators of the PGARA allocator.



NISHANTH KUMAR is pursuing the B.Tech. degree in Computer Science and Engineering (Artificial Intelligence and Machine Learning) at the Vellore Institute of Technology, Vellore, India. In this work he assisted with the documentation and with testing the web interface.